UVSQ & LEDGER DONJON

Master thesis

Construction of a SNARK from the FRI protocol

Mareau Quentin

Supervised by Yannick Seurin

Year 2026

Abstract

This report has been written during a cryptography intership at Ledger where the main goal was studying how realistic implementing a Verifier on their device was. I firstly learned about SNARK and the many existing implmentations, what they offers and what was the obstacles on the focused implementation. It led me to study the memory necessary for such a Verifier program and how to manage it. For helping me to really understand all the parameters involved in studied implementations, I wrote this report about the construction of a SNARK based on ALI and FRI (close to risc0 approach). Thus, this work is the side report of this work about the implementation of a Verifier in a constrained system.

CONTENTS

1 INTRODUCTION

The **SNARKs**, for *Succinct Non-interactive ARgument of Knowledge*, formalized by [BCCT11], address a delegated-computation problem. More specifically, the classical model considers a local device, often less powerful, which asks a remote device, often more powerful, to execute a program and send back the result. In this context, the first device would like to be convinced that the result it receives is indeed the result of the agreed program, executed on the agreed inputs. The goal is to prevent two phenomena:

- unexpected computation errors by the remote device;
- lies from a malicious remote device.

Without any particular efficiency constraint, the simplest answer is for the local device to recompute the result itself. SNARKs address additional constraints by imposing the following model:

1. The remote device, called the Prover, performs the intended computation if it is honest and generates a proof from this computation. This proof must be small compared with the computation performed and is transmitted in addition to the result. It is useful to note that this proof depends on the program executed by the Prover, on the inputs, and therefore indirectly on the output.
2. The local device, called the Verifier, receives the result and the proof, then checks the validity of this pair at low cost.

A *zero-knowledge* aspect is often added to these constraints, so that the proof reveals no superfluous information about the program's *private* inputs. A classical example is based on the following assertion: "For a fixed hash function and a public hash h, the Prover knows an x such that $H(x) = h$." The Prover's goal is then to run an implementation of the hash function and send a proof that a correct execution did return h, without providing any other information about x. This property is often highlighted in concrete application projects such as confidential identity proofs, private transactions, or cloud *verifiable computing*.

This field has evolved substantially over the last ten years, notably with the arrival of proof systems such as Groth16 [Gro16] and Plonk [GWC19]. These proof systems offer low proof-generation and verification times, as well as constant and very small proof sizes; they therefore provide an excellent answer to the problems stated above. However, they have two major drawbacks:

- they rely on elliptic-curve-related assumptions, which makes them vulnerable to quantum-computer attacks;
- they depend on a *trusted setup*, meaning that the Prover and the Verifier must first agree on common parameters for the system. If some secret data associated with this generation, often grouped under the name *toxic waste*, were kept, the security of the protocol could be compromised.

This is why **STARKs**, for *Scalable Transparent ARgument of Knowledge*, then appeared in [BSBHR18b]. In their non-interactive version, they can be seen as transparent and scalable SNARKs, meaning in particular that they impose quasi-linear proving time on the Prover. These proof systems do not require a *trusted setup* and often rely on assumptions compatible with post-quantum security. However, these advantages come with a major drawback: proof size is no longer constant, and proofs are much larger than for Plonk and Groth16.

This report presents a construction of a STARK based on an *Interactive Oracle Proof of Proximity* (IOPP) named FRI, for *Fast Reed-Solomon Interactive Oracle Proof of Proximity*. This protocol lies at the heart of many proof systems based on hash functions and error-correcting codes. We will use it as an opportunity to study its operation in detail, its security, and its different variants.

In the construction of STARKs, an essential step consists in reformulating the execution of a program in algebraic form, then reducing its verification to a mathematical problem: this is *arithmetization*. In our context, we associate to the computation a function f on a domain $\mathcal{L}$ with values in a finite field $\mathbb{F}$, then fix an integer $d > 0$ so as to obtain the following property: if f is sufficiently close to a polynomial of degree $< d$, for the Hamming distance, then the program was executed as intended and did return the announced value, except with negligible probability. This is where FRI is useful: it makes it possible to test efficiently that a function $f : \mathcal{L} \longrightarrow \mathbb{F}$ is close to a word of a Reed-Solomon code. The construction of our STARK can then be summarized by the following figure:

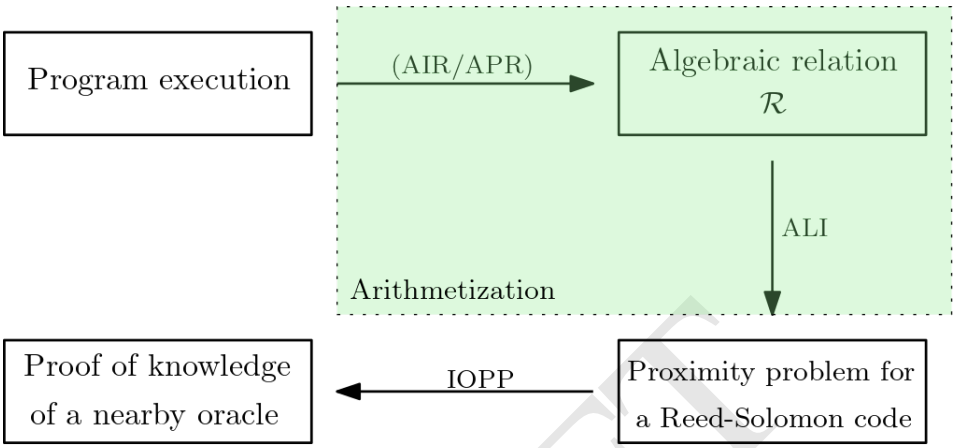


Figure 1: Construction of an IOPP-based STARK

2 PRELIMINARIES

2.1 Reed-Solomon codes

[TO BE REWORKED WITH [Sta23]]

Definition 2.1. Let $\mathbb{F}$ be a finite field and let d, n be integers such that $d \leq n$. A linear code $\mathcal{C}$ of length n and dimension d over $\mathbb{F}$ is a vector subspace of $\mathbb{F}^n$ of dimension d. The minimum distance of the code is defined by

$$l = l(\mathcal{C}) = \min_{u \in \mathcal{C} \setminus \{0\}} w(u) \in \mathbb{N} \qquad \text{where } w \text{ denotes the Hamming weight.}$$

Such a code is denoted $[n, d, l]$.

Definition 2.2. We also define:

- the Hamming distance by $d(u, v) = w(u - v)$;
- the relative Hamming distance by $\Delta(u, v) = \frac{1}{n} w(u - v) \in [0, 1]$;
- the relative minimum distance of the code by $\mu = \mu(\mathcal{C}) = \frac{l(\mathcal{C})}{n} \in]0, 1]$;
- the code rate by $\rho = \frac{d}{n} \in [0, 1]$.

Proposition 2.1 (Singleton bound)**.** *Let $\mathcal{C}$ be an $[n, d, l]$-code. Then $l \leq n - d + 1$.*
A code that reaches this bound is called MDS (Maximum Distance Separable).

Proof. Suppose that $d > n - l + 1$. Then $|\mathcal{C}| > |\mathbb{F}|^{n-l+1}$ and, by the pigeonhole principle, there necessarily exist two distinct vectors u, v that coincide on at least $n - l + 1$ coordinates. We therefore have $d(u, v) \leq l - 1$, which is absurd. □

Definition 2.3. Let $\mathcal{C}$ be an $[n, d, l]$-code. For $u \in \mathbb{F}^n$ and $\delta \in [0, 1]$, the decoding list is defined by

$$\text{List}[u, \mathcal{C}, \delta] = \{c \in \mathcal{C},\ \Delta(c, u) \leq \delta\} = \{c \in \mathcal{C},\ d(c, u) \leq n\delta\}.$$

[THIS PART: TO BE REVISED]

By the triangle inequality, we immediately observe that if $\delta < \frac{\mu}{2}$, then, for every $u \in \mathbb{F}^n$, the set $\text{List}[u, \mathcal{C}, \delta]$ contains at most one element. In other words, every element of $\mathbb{F}^n$ has at most one representative of the code at relative distance δ; this is called the **unique decoding regime**. One may then ask whether, for an arbitrary value of δ, it is possible to estimate the size of $\text{List}[u, \mathcal{C}, \delta]$.

Lemma 2.2. *Let $\mathcal{C}$ be an $[n, d, l]$-code, $u \in \mathbb{F}^n$ and $\delta \in [0, 1]$ such that $(1 - \delta)^2 > 1 - \mu$. Then*

$$|\text{List}[u, \mathcal{C}, \delta]| \leq \frac{\mu}{(1 - \delta)^2 - (1 - \mu)}.$$

Proof. We apply Theorem 3.3 of [Gur07] with $e = n\delta$. □

Theorem 2.3 (Strong Johnson bound)**.** *Let $\mathcal{C}$ be an $[n, d, l]$-code and $u \in \mathbb{F}^n$. Let $\varepsilon > 0$. If*

$$0 < \delta < 1 - \sqrt{1 - \mu + \mu\varepsilon},$$

then

$$|\text{List}[u, \mathcal{C}, \delta]| \leq \frac{1}{\varepsilon}.$$

Proof. We first observe that $1 - \delta$ and $\sqrt{1 - \mu + \mu\varepsilon}$ are positive. Thus,

$$\delta < 1 - \sqrt{1 - \mu + \mu\varepsilon} \iff (1 - \delta)^2 > 1 - \mu + \mu\varepsilon \iff (1 - \delta)^2 - (1 - \mu) > \mu\varepsilon.$$

We therefore have $(1-\delta)^2 > 1-\mu$, and the preceding lemma gives

$$|\mathrm{List}[u,\mathcal{C},\delta]| \leq \frac{\mu}{(1-\delta)^2-(1-\mu)} \leq \frac{\mu}{\mu\varepsilon} \leq \frac{1}{\varepsilon}.$$

□

Corollary 2.4 (Weak Johnson bound)**.** *Let $\mathcal{C}$ be an $[n,d,l]$-code, $u \in \mathbb{F}^n$ and $\delta \in \left]0, 1-\sqrt{1-\mu}\right[$. Then*

$$|\mathrm{List}[u,\mathcal{C},\delta]| \leq \frac{1}{\varepsilon_\delta} \qquad \textit{where} \quad \varepsilon_\delta = 2\sqrt{1-\mu}\left(1-\sqrt{1-\mu}-\delta\right).$$

Proof. We wish to apply the strong version of the Johnson bound with

$$\varepsilon_\delta = 2\sqrt{1-\mu}\left(1-\sqrt{1-\mu}-\delta\right).$$

It suffices to verify that, for $0 < \delta < 1-\sqrt{1-\mu}$, we have

$$0 < \varepsilon_\delta < (1-\delta)^2 - (1-\mu).$$

First,

$$\delta < 1-\sqrt{1-\mu} \iff 1-\sqrt{1-\mu}-\delta > 0,$$

hence $\varepsilon_\delta > 0$.

Next, we observe that

$$\begin{aligned}(1-\delta)^2-(1-\mu) &= (1-\delta)^2 - \left(\sqrt{1-\mu}\right)^2 \\ &= \left(1-\delta+\sqrt{1-\mu}\right)\left(1-\sqrt{1-\mu}-\delta\right).\end{aligned}$$

Since $\delta < 1-\sqrt{1-\mu}$, we have

$$1-\delta+\sqrt{1-\mu} > 2\sqrt{1-\mu},$$

and therefore

$$(1-\delta)^2-(1-\mu) > 2\sqrt{1-\mu}\left(1-\sqrt{1-\mu}-\delta\right) = \varepsilon_\delta.$$

□

Definition 2.4. Let $n > d$ be two nonzero integers. The Reed-Solomon code over $\mathbb{F}$, with evaluation domain $\mathcal{L}$ of size n and degree d, is the linear code

$$\mathrm{RS}[\mathbb{F},\mathcal{L},d] = \left\{\overline{f} : \mathcal{L} \longrightarrow \mathbb{F} \mid f \in \mathbb{F}^{<d}[X]\right\}.$$

Proposition 2.5. *Let $d < n$ be two nonzero integers. Then $RS[\mathbb{F},\mathcal{L},d]$ is an $[n,d,l]$-code with $l = n-d+1$. In other words, $RS[\mathbb{F},\mathcal{L},d]$ is an MDS code.*

Proof. The dimension of the code is that of $\mathbb{F}^{<d}[X]$, hence d. Next, every nonzero $f \in \mathbb{F}^{<d}[X]$ has at most $d-1$ roots, so

$$w(f) = |\{x \in \mathcal{L},\ f(x) \neq 0\}| \geq n-(d-1).$$

We deduce that $l \geq n-d+1$ and, by the Singleton bound, finally $l = n-d+1$. □

For a code $\mathrm{RS}[\mathbb{F},\mathcal{L},d]$, we have in particular the implication

$$\delta \leq \frac{1-\rho}{2} \implies \delta < \frac{\mu}{2}.$$

We will keep this sufficient condition for the characterization of the unique decoding regime.

$$1-\mu = \frac{n-(n-d+1)}{n} = \frac{d-1}{n}.$$

Thus, when $n \gg d$, one can approximate

$$1-\mu \approx \frac{d}{n} = \rho.$$

The Johnson bound is then reformulated as follows.

Corollary 2.6. *Let $RS[\mathbb{F},\mathcal{L},d]$ be such that $n \gg d$, let $u \in \mathbb{F}^n$ and let $\delta \in]0,1-\sqrt{\rho}[$. Then*

$$|\mathrm{List}[u,RS[\mathbb{F},\mathcal{L},d],\delta]| \leq \frac{1}{\varepsilon_\delta} \qquad \textit{where} \quad \varepsilon_\delta = 2\sqrt{\rho}\,(1-\sqrt{\rho}-\delta)\,.$$

2.2 Interactive Oracle Proofs and IOPs of Proximity

Definition 2.5. A **relation** is a subset $\mathcal{R} \subseteq \{0,1\}^* \times \{0,1\}^*$. If $(\mathsf{x},\mathsf{w}) \in \mathcal{R}$, we say that x is an *instance* and that w is a *witness* for x. For $\mathsf{x} \in \{0,1\}^*$, we denote $\mathcal{R}|_{\mathsf{x}} = \{\mathsf{w} \in \{0,1\}^* \mid (\mathsf{x},\mathsf{w}) \in \mathcal{R}\}$.

Definition 2.6. The **language** associated with a relation $\mathcal{R}$ is

$$\mathcal{L}(\mathcal{R}) = \{\mathsf{x} \in \{0,1\}^* \mid \exists \mathsf{w} \in \{0,1\}^*,\ (\mathsf{x},\mathsf{w}) \in \mathcal{R}\}\,.$$

Definition 2.7. An interactive protocol is said to be *public-coin* if all messages sent by the Verifier are uniformly random challenges, independent of the messages previously sent by the Prover.

Definition 2.8. An *interactive oracle proof*, abbreviated as **IOP**, for a relation $\mathcal{R}$, with soundness error $\varepsilon : \{0,1\}^* \to [0,1]$ and number of rounds $k : \{0,1\}^* \to \mathbb{N}$, is a pair of probabilistic polynomial-time interactive algorithms (P,V) such that:

0. **Setup:** for every honest input $(\mathsf{x},\mathsf{w}) \in \mathcal{R}$, the Prover receives (x,w) and the Verifier receives x;
1. **Interaction:** at each round $i \in \{1,\dots,k(\mathsf{x})\}$, the Verifier sends a message α_i and the Prover sends an *oracle message* m_i, which the Verifier does not read in full;
2. **Queries:** the Verifier queries the oracle messages sent by the Prover at some locations. At the end of the protocol, the Verifier returns accept or reject;
3. **Completeness:** for every $(\mathsf{x},\mathsf{w}) \in \mathcal{R}$,
$$\mathbb{P}\left[\langle P(\mathsf{x},\mathsf{w}),\, V(\mathsf{x})\rangle = \mathsf{accept}\right] = 1;$$
4. **Soundness:** for every $\mathsf{x} \notin \mathcal{L}(\mathcal{R})$ and every possibly unbounded malicious Prover $\tilde{P}$, we have
$$\mathbb{P}\left[\left\langle \tilde{P}(\mathsf{x}),\, V(\mathsf{x})\right\rangle = \mathsf{accept}\right] \leq \varepsilon(\mathsf{x}).$$

We denote by $p(\mathsf{x}) = \sum_{i=1}^{k(\mathsf{x})} |m_i|$ the *proof size* and by $q(\mathsf{x})$ the total number of entries read by V, called the *query complexity*.

Remark. An IOP can be viewed as a compromise between an interactive proof and a PCP (see [CY24]): interaction is preserved, but the Prover's long messages are treated as oracles that the Verifier probes at only a small number of positions. These queries are generally adaptive and may depend on the entire preceding transcript. In what follows, we are interested exclusively in public-coin IOPs, a condition which, as indicated above, is necessary for compilation by BCS/Fiat-Shamir into a non-interactive argument.

Definition 2.9 (Argument of knowledge)**.** An IOP (P,V) for a relation $\mathcal{R}$ is an *argument of knowledge* with error (or *knowledge-soundness error*) $\mathsf{e} : \{0,1\}^* \longrightarrow [0,1]$ if there exists a probabilistic polynomial-time extractor E such that, for every instance x and every malicious Prover $\tilde{P}$, we have

$$\mathbb{P}\left[(\mathsf{x}, E^{\tilde{P}}(\mathsf{x})) \in \mathcal{R}\right] \geq \mathbb{P}\left[\left\langle \tilde{P}(\mathsf{x}),\, V(\mathsf{x})\right\rangle = \mathsf{accept}\right] - \mathsf{e}(\mathsf{x}),$$

where $E^{\tilde{P}}$ denotes E with access to the transcript of $\tilde{P}$ at every stage of the protocol (but not to its auxiliary inputs or to its internal randomness).

Remark. For $\mathsf{x} \notin \mathcal{L}(\mathcal{R})$, we observe that if our IOP is an argument of knowledge, then

$$\mathbb{P}\left[\left\langle \tilde{P}(\mathsf{x}),\, V(\mathsf{x})\right\rangle = \mathsf{accept}\right] \leq \mathsf{e}(\mathsf{x}).$$

Thus, this property strengthens the usual soundness: a Prover who manages to convince the Verifier must in reality *know* a witness, in the sense that a witness can be extracted from him. This notion is essential for SNARKs, which are by definition *arguments of knowledge*.

Definition 2.10 (State function)**.** A *state function* for an IOP (P,V) is a deterministic function, possibly not computable in polynomial time,

$$\mathsf{State} : \{0,1\}^* \times \mathcal{T} \longrightarrow \{0,1\},$$

where $\mathcal{T}$ denotes the set of transcripts, partial or complete, of (P,V), satisfying the following conditions:

1. **Empty transcript:** $\mathsf{State}(\mathsf{x}, \varnothing) = 1 \iff \mathsf{x} \in \mathcal{L}(\mathcal{R})$;
2. **Prover interaction:** if tr is a partial transcript for which the Prover is about to play and $\mathsf{State}(\mathsf{x}, \mathsf{tr}) = 0$, then for every message π that can be sent by the Prover,
$$\mathsf{State}(\mathsf{x}, \mathsf{tr} \,\|\, \pi) = 0;$$
3. **Complete transcript:** if tr is a complete transcript and $\mathsf{State}(\mathsf{x}, \mathsf{tr}) = 0$, then the Verifier returns reject;
4. **Default behavior:** for every transcript tr, complete or partial, and every message π, if $\mathsf{State}(\mathsf{x}, \mathsf{tr}) = 1$, then $\mathsf{State}(\mathsf{x}, \mathsf{tr} \,\|\, \pi) = 1$.

Definition 2.11 (Round-by-round soundness)**.** We say that a k-round public-coin IOP (P,V) has vector-valued *round-by-round soundness* with error $(\varepsilon_1, \ldots, \varepsilon_k) \in [0,1]^k$ if there exists a state function State for (P,V) such that, for every instance $\mathsf{x} \notin \mathcal{L}(\mathcal{R})$, every index $i \in \{1, \ldots, k\}$ and every transcript tr of the first i rounds for which the Verifier is about to play and $\mathsf{State}(\mathsf{x}, \mathsf{tr}) = 0$, we have

$$\mathbb{P}_{\sigma}\left[\mathsf{State}(\mathsf{x}, \mathsf{tr} \,\|\, \sigma) = 1\right] \leq \varepsilon_i,$$

where σ is the Verifier's challenge at round $i+1$, drawn uniformly.

Remark. This formulation, due to [Sta23], refines the usual notion by assigning a specific error to each round. It is central to the analysis of protocols made non-interactive by Fiat-Shamir: the conditions imposed by the state function guarantee that, at each round, producing a valid malicious transcript is an exceptional event, whose probability is precisely controlled by ε_i.
We also observe that, for round-by-round soundness given by the ε_i, one can easily upper-bound a global soundness error by $\sum_{i=1}^{k} \varepsilon_i$.

An important variant of IOPs is given by *IOPs of Proximity*, abbreviated as **IOPP**. In the relation-based view of IOPs, one can regard an IOPP as specialized to proximity relations: for a linear code $\mathcal{C}$ and a proximity parameter δ, the associated assertion is that an oracle $f_0 \in \mathbb{F}^{\mathcal{L}}$ is δ-close to $\mathcal{C}$ in relative Hamming distance. The definition below packages these thresholds in a single soundness function $\varepsilon(\delta)$. The case of practical interest to us is the one where $\mathcal{C} = \mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$: the goal is then to test that an oracle f_0 is close to the evaluation of a polynomial of degree $< d$, which is precisely the task performed by FRI.

Definition 2.12 (IOPP)**.** Let $\mathcal{C} \subseteq \mathbb{F}^{\mathcal{L}}$ be a linear code. A k-round *IOP of Proximity* for $\mathcal{C}$, with soundness error $\varepsilon : [0,1] \to [0,1]$, is a $(k+1)$-round IOP (P, V) satisfying:

1. **First-message format:** the first message sent by the Prover is a word $f_0 \in \mathbb{F}^{\mathcal{L}}$, transmitted as an oracle to the Verifier;
2. **Completeness:** if $f_0 \in \mathcal{C}$, then
$$\mathbb{P}\left[\langle P(f_0, \mathcal{C}),\ V(\mathcal{C})\rangle = \mathsf{accept}\right] = 1;$$
3. **Soundness:** for every $\delta \in [0,1]$, for every $f_0 \in \mathbb{F}^{\mathcal{L}}$ such that $\Delta(f_0, \mathcal{C}) \geq \delta$ and for every malicious Prover $\tilde{P}$,
$$\mathbb{P}\left[\langle \tilde{P}(f_0, \mathcal{C}),\ V(\mathcal{C})\rangle = \mathsf{accept}\right] \leq \varepsilon(\delta).$$

Remark. The notions of proof size, query complexity, and round-by-round soundness carry over directly to IOPPs. In what follows, we will denote by $\mathrm{IOPP}(f_0, \mathcal{C}) \to \{\mathsf{accept}, \mathsf{reject}\}$ the execution of an IOPP with initial word f_0 and target code $\mathcal{C}$. The construction of a STARK then follows the standard pattern: an IOP reduces the arithmetic instance to a proximity test to a Reed-Solomon code, which is then solved by an IOPP, typically FRI, before being compiled into a non-interactive argument by Merkle commitment and the BCS transformation.

2.3 SNARKs and BCS transformation

The acronyms **SNARK** and **STARK** highlight complexity properties of a non-interactive *argument of knowledge*. We adopt here the following convention, compatible with the standard terminology of succinct arguments [Gro16] and with the literature derived from IOPs [BSCS16, BSBHR18b]. The properties of completeness, soundness, and extractability are the non-interactive analogues of those introduced for IOPs. Throughout the rest of this report, λ denotes the security parameter, x the public instance, w a witness for x, and $T := T(\mathsf{x}, \mathsf{w})$ denotes a measure of the size of the proven computation.

Definition 2.13 (SNARK)**.** Let $\mathcal{R}$ be a relation. A **SNARK** (*Succinct Non-interactive ARgument of Knowledge*) for $\mathcal{R}$ is a triple of algorithms:

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$: takes as input the security parameter λ and returns public parameters pp;
- $\pi \leftarrow \mathsf{Prove}(\mathsf{pp}, \mathsf{x}, \mathsf{w})$: takes as input pp, an instance $\mathsf{x} \in \mathcal{L}(\mathcal{R})$ and $\mathsf{w} \in \mathcal{R}|_{\mathsf{x}}$, and returns a *proof* $\pi \in \{0,1\}^*$;
- $b \leftarrow \mathsf{Verify}(\mathsf{pp}, \mathsf{x}, \pi)$: takes as input pp, an instance x and a word π, and returns accept or reject.

These algorithms must satisfy completeness, knowledge soundness (which implies computational soundness), and **succinctness**: for every honest input $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}$ of computation size T,

$$|\pi| \leq \mathsf{poly}(\lambda, \log T) \qquad \text{and} \qquad \mathrm{Time}\big(\mathsf{Verify}(\mathsf{pp}, \mathsf{x}, \pi)\big) = O(|\mathsf{x}|, \mathsf{poly}(\lambda, \log T)),$$

where $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$.

Definition 2.14 (STARK)**.** A **STARK** (*Scalable Transparent ARgument of Knowledge*) for $\mathcal{R}$ is a SNARK for $\mathcal{R}$ which additionally satisfies:

1. **Transparency:** the public parameters pp are generated only from public randomness and contain no secret of the *toxic waste* type;
2. **Scalability:** for every honest input $(\mathsf{x},\mathsf{w}) \in \mathcal{R}$ of computation size T,

$$\mathrm{Time}\big(\mathsf{Prove}(\mathsf{pp},\mathsf{x},\mathsf{w})\big) \leq \widetilde{O}(T)\mathsf{poly}(\lambda).$$

Remark. The *scalable* property therefore strengthens the *succinct* property by adding a constraint on the Prover's running time. In this convention, a STARK is essentially a transparent SNARK whose proving time is quasi-linear in the size of the computation. Conversely, a transparent non-interactive argument satisfying these complexity bounds belongs, in the usual sense, to the family of STARKs.

Provided that proof sizes and verification times are sufficiently bounded, we observe that the notion of IOP, with argument of knowledge, studied above seems particularly well suited to the construction of SNARKs. However, it is first necessary to make this kind of protocol *non-interactive*:

Definition 2.15 (Non-interactive Oracle Proof)**.** A *non-interactive oracle proof* for a relation $\mathcal{R}$, with soundness error $\varepsilon : \{0,1\}^* \times \mathbb{N}^* \to [0,1]$, is a pair of probabilistic polynomial-time algorithms (P,V) such that:

1. for every honest input $(\mathsf{x},\mathsf{w}) \in \mathcal{R}$, the Prover receives (x,w) and the Verifier receives x;
2. **Non-interactivity:** for every honest input $(\mathsf{x},\mathsf{w}) \in \mathcal{R}$, the Prover simulates an IOP with Q queries using a random oracle H and produces a single message π which he sends to the Verifier;
3. the algorithm $V^H(\mathsf{x},\pi)$ returns either accept or reject;
4. **Completeness:** for every $(\mathsf{x},\mathsf{w}) \in \mathcal{R}$,

$$\mathbb{P}\left[V^H(\mathsf{x},\pi) = \mathsf{accept} \mid H \leftarrow \mathcal{U}(\lambda),\ \pi \leftarrow P^H(\mathsf{x},\mathsf{w})\right] = 1;$$

5. **Soundness:** for every $\mathsf{x} \notin \mathcal{L}(\mathcal{R})$ and every malicious Prover $\tilde{P}$ making Q queries, possibly unbounded, we have

$$\mathbb{P}\left[V^H(\mathsf{x},\pi) = \mathsf{accept} \mid H \leftarrow \mathcal{U}(\lambda),\ \pi \leftarrow \tilde{P}^H(\mathsf{x})\right] \leq \varepsilon(\mathsf{x},Q).$$

We will say that this protocol is an argument of knowledge with error $\mathsf{e} : \{0,1\}^* \times \mathbb{N}^* \longrightarrow [0,1]$ if there exists a probabilistic polynomial-time extractor E such that, for every instance x and every malicious Prover $\tilde{P}$ making Q queries, we have

$$\mathbb{P}\left[(\mathsf{x},E^{\tilde{P}}(\mathsf{x})) \in \mathcal{R}\right] \ \geq\ \mathbb{P}\left[V^H(\mathsf{x},\pi) = \mathsf{accept} \mid H \leftarrow \mathcal{U}(\lambda),\ \pi \leftarrow \tilde{P}^H(\mathsf{x})\right] - \mathsf{e}(\mathsf{x},Q).$$

Remark. Above, the notation $\mathcal{U}(\lambda)$ denotes the uniform distribution over the set of functions $H : \{0,1\}^* \longrightarrow \{0,1\}^\lambda$ and therefore $H \leftarrow \mathcal{U}(\lambda)$ denotes a random oracle.

The naive approach is the direct use of the Fiat-Shamir procedure used in classical interactive protocols. In this setting, one fixes a random oracle H and each step proceeds as follows:

- the Prover has constructed his message to the Verifier m_{i-1} at the previous step;
- he simulates the random challenge $\alpha_i = H(\mathsf{x} \,\|\, \alpha_{i-1} \,\|\, m_{i-1})$, or $\alpha_1 = H(\mathsf{x})$ if $i = 1$;

- he constructs m_i as he would have done during the interactive protocol.

The final proof provided by the Prover to the Verifier is then $\pi = (m_1, \dots, m_k)$. The Verifier can then reconstruct the α_i and perform his usual verification.

This approach does not adapt directly to IOPs because, in this model, the Verifier cannot read the m_i entirely. For this reason, [BSCS16] introduced a refinement of Fiat-Shamir adapted to IOPs, later called the *BCS transformation*. This method consists in adapting the philosophy of Fiat-Shamir both for the construction of oracle messages and for the Verifier's queries. In particular, it uses the notion of *Merkle trees*, which we present in section 3.1.

We now present the main result, from [BSCS16, Theorem 7.1] and [BGK+23, Theorem 3.15], making it possible to justify the construction of SNARKs from IOPs. We refer to these articles for a more formal and precise description of the technique.

Theorem 2.7 (Properties of BCS)**.** *Let $\mathcal{R}$ be a relation and let (P,V) be a public-coin IOP with $k := k(\mathsf{x})$ rounds, $q := q(\mathsf{x})$ queries, round-by-round soundness $(\varepsilon_1, \dots, \varepsilon_k)$, and proof size $p := p(\mathsf{x})$. We also denote by T_P and T_V the respective time complexities of the Prover and the Verifier, and by Q the maximum number of queries simulated by an adversary. Then the non-interactive protocol obtained from the BCS transformation, defined for a security parameter λ, satisfies:*

(i) ***Soundness.*** $\varepsilon'(\mathsf{x}, Q, \lambda) = Q \cdot \max_i \varepsilon_i + 3(Q^2+1) \cdot 2^{-\lambda}$;

(ii) ***Proof size.*** $p'(\mathsf{x}, \lambda) = (k + q \cdot (\lceil \log_2 p(\mathsf{x}) \rceil + 2) + 1) \cdot \lambda$;

(iii) ***Proving time.*** $T'_P(\mathsf{x}, \lambda) = \mathsf{poly}(\lambda) \cdot O(k+p) + T_P + T_V$;

(iv) ***Verification time.*** $T'_V(\mathsf{x}, \lambda) = \mathsf{poly}(\lambda) \cdot O(k+q) + T_V$.

Moreover, if (P,V) is an argument of knowledge with error e*, then so is its non-interactive version,*

$$\mathsf{e}'(\mathsf{x}, Q, \lambda) = Q \cdot \mathsf{e}(\mathsf{x}) + 3(Q^2+1) \cdot 2^{-\lambda}.$$

As announced, this result allows us to restrict our construction to that of an IOP with argument of knowledge satisfying the complexity constraints of SNARKs. In that case, one must ensure that the relation associated with our IOP establishes a link with the correct execution of a program. This will be studied in section 4.

2.4 Polynomial Commitment Schemes

A *Polynomial Commitment Scheme* (PCS) allows a Prover to commit to a polynomial of bounded degree and then convince a Verifier that a public tuple of values indeed corresponds to the evaluations of the committed polynomial at a public tuple of points, without revealing the entire polynomial. This cryptographic primitive first appeared in [KZG10] in 2010 and has since been refined into various adaptations, each suited to its own context. In the setting of interest to us, we adopt a *batched* definition inspired by that of [Fu25], in which the evaluation algorithm is a public-coin interactive protocol allowing several evaluations to be queried simultaneously.

Definition 2.16. Let $\mathbb{F}$ be a field and let $d, m_{\max} \in \mathbb{N}$, where d is an upper bound on the degree of the accepted polynomials and $m_{\max}$ is an upper bound on the number of simultaneous evaluation queries. A *polynomial commitment scheme* over $\mathbb{F}$ is a quadruple of probabilistic polynomial-time algorithms

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, d, m_{\max})$: takes as input the security parameter λ, the bound d, and the bound $m_{\max}$, and returns public parameters pp.

- $C \leftarrow \mathsf{Commit}(\mathsf{pp}, d, f(X))$: takes as input pp, d, and a polynomial $f \in \mathbb{F}^{<d}[X]$, and returns a commitment C.
- $b \in \{0,1\} \leftarrow \mathsf{Open}(\mathsf{pp}, d, C, f(X))$: takes as input pp, d, C, and $f(X)$, and returns 1 if $f(X)$ is accepted by the opening relation associated with C, and 0 otherwise.
- $b \in \{0,1\} \leftarrow \mathsf{Eval}(\mathsf{pp}, d, C, \Omega, \mathbf{y}; f(X))$: denotes a public-coin interactive protocol

$$\langle P(\mathsf{pp}, d, C, \Omega, \mathbf{y}, f(X)),\ V(\mathsf{pp}, d, C, \Omega, \mathbf{y}) \rangle .$$

Here we denote

$$\Omega = (z_1, \ldots, z_m) \in \mathbb{F}^m \qquad \text{with} \qquad m \leq m_{\max},$$

where the z_i are pairwise distinct, and

$$\mathbf{y} = (y_1, \ldots, y_m) \in \mathbb{F}^m.$$

The Prover and the Verifier have as common inputs pp, d, C, Ω, and $\mathbf{y}$. The Prover additionally knows a polynomial $f(X)$ such that $\mathsf{Open}(\mathsf{pp}, d, C, f(X)) = 1$. At the end of the protocol, the Verifier returns a bit b such that $b = 1$ if the claim

$$f(z_i) = y_i \qquad \text{for every } i \in \{1, \ldots, m\}.$$

is accepted, and $b = 0$ otherwise.

This quadruple must satisfy the following properties:

1. **Completeness.** For all $\lambda, d, m_{\max} \in \mathbb{N}$, every polynomial $f \in \mathbb{F}^{<d}[X]$, and every admissible tuple

$$\Omega = (z_1, \ldots, z_m) \in \mathbb{F}^m$$

of pairwise distinct points, with $m \leq m_{\max}$, we have

$$\mathbb{P}\left[\langle P(\mathsf{pp}, d, C, \Omega, \mathbf{y}, f(X)),\ V(\mathsf{pp}, d, C, \Omega, \mathbf{y}) \rangle = 1 \;\middle|\; \begin{array}{c} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, d, m_{\max}) \\ C \leftarrow \mathsf{Commit}(\mathsf{pp}, d, f(X)) \\ \mathbf{y} = (f(z_1), \ldots, f(z_m)) \end{array} \right] = 1.$$

2. **Binding.** For every probabilistic polynomial-time adversary $\mathcal{A}$, we have

$$\mathbb{P}\left[b_1 = b_2 = 1 \wedge f_1(X) \neq f_2(X) \;\middle|\; \begin{array}{c} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, d, m_{\max}) \\ (C, d, f_1(X), f_2(X)) \leftarrow \mathcal{A}(\mathsf{pp}) \\ b_1 \leftarrow \mathsf{Open}(\mathsf{pp}, d, C, f_1(X)) \\ b_2 \leftarrow \mathsf{Open}(\mathsf{pp}, d, C, f_2(X)) \end{array} \right] \leq \mathrm{negl}(\lambda).$$

3. **Knowledge extractability.** The PCS satisfies knowledge extractability if the protocol Eval is an argument of knowledge for the relation

$$\mathcal{R}_{\mathsf{Eval}} = \left\{ \big((\mathsf{pp}, d, C, \Omega, \mathbf{y}), f(X)\big) \;\middle|\; \begin{array}{c} \Omega = (z_1, \ldots, z_m) \in \mathbb{F}^m \text{admissible} \\ \mathsf{Open}(\mathsf{pp}, d, C, f(X)) = 1 \\ f(z_i) = y_i \text{ for every } i \in \{1, \ldots, m\} \end{array} \right\}.$$

In other words, every Prover who convinces the Verifier in Eval is able to construct $f \in \mathbb{F}^{<d}[X]$ in polynomial time such that $\mathsf{Open}(\mathsf{pp}, d, C, f(X)) = 1$ and

$$f(z_i) = y_i \qquad \text{for every } i \in \{1, \ldots, m\}.$$

Remark. The algorithm Open should be understood as the opening relation specified by the considered scheme. In classical schemes, this relation may express that C is an exact commitment to f. We will se that in proximity-based schemes, it may instead express that the object bound by C is sufficiently close to the evaluation of f over a prescribed domain. The binding property below is what ensures that this opening relation cannot accept two distinct polynomials for the same commitment, except with negligible probability.
Also, in the context of zero-knowledge proofs, one may also require that the protocol satisfy a *hiding* property, meaning that no information about the polynomial is revealed through the interactions.

Proposition 2.8 (Opening consistency)**.** *Consider a PCS over* $\mathbb{F}$*. For every probabilistic polynomial-time adversary* $\mathcal{A}$*, we have*

$$\mathbb{P}\left[\begin{array}{c}\left((\mathsf{pp},d,C,\Omega_1,\mathbf{y}_1),f_1(X)\right)\in\mathcal{R}_{\mathsf{Eval}}\\ \left((\mathsf{pp},d,C,\Omega_2,\mathbf{y}_2),f_2(X)\right)\in\mathcal{R}_{\mathsf{Eval}}\\ f_1(X)\neq f_2(X)\end{array}\middle|\begin{array}{c}\mathsf{pp}\leftarrow\mathsf{Setup}(1^\lambda,d,m_{\max})\\ (C,\Omega_1,\mathbf{y}_1,\Omega_2,\mathbf{y}_2,f_1(X),f_2(X))\leftarrow\mathcal{A}(\mathsf{pp})\end{array}\right]\leq \mathrm{negl}(\lambda).$$

Proof. Suppose that the considered event occurs. By definition of the relation $\mathcal{R}_{\mathsf{Eval}}$, we then have

$$\mathsf{Open}(\mathsf{pp},d,C,f_1(X))=1 \qquad \text{and} \qquad \mathsf{Open}(\mathsf{pp},d,C,f_2(X))=1,$$

as well as

$$f_1(X)\neq f_2(X).$$

We then construct a probabilistic polynomial-time adversary $\mathcal{A}'$ against the binding property as follows: on input pp, it runs $\mathcal{A}(\mathsf{pp})$, obtains

$$(C,\Omega_1,\mathbf{y}_1,\Omega_2,\mathbf{y}_2,f_1(X),f_2(X)),$$

then returns

$$(C,d,f_1(X),f_2(X)).$$

As soon as the event above occurs for $\mathcal{A}$, the adversary $\mathcal{A}'$ therefore produces two distinct polynomials $f_1(X)$ and $f_2(X)$ such that

$$\mathsf{Open}(\mathsf{pp},d,C,f_1(X))=\mathsf{Open}(\mathsf{pp},d,C,f_2(X))=1.$$

This contradicts the binding property of the PCS. The probability that the considered event occurs is therefore negligible in λ. □

We will see in a later part that it is possible to *summarize* the execution of a program by a few polynomials linked by algebraic properties. Thus, proving knowledge of these particular polynomials amounts to proving the existence of a correct execution of the program. It is in this sense that PCS form a fundamental part of the construction of arguments of knowledge.
There are many ways to construct such protocols relying on different cryptographic primitives. For this report, we focus on PCS based on IOP/IOPP, whose operation can be heuristically summarized by the following property:

$$\Big(f\in\mathbb{F}^{<d}[X] \quad \text{and} \quad f(z)=y\Big) \qquad \Longleftrightarrow \qquad \text{The function } x\mapsto\frac{f(x)-y}{x-z} \text{ is a polynomial of } \mathbb{F}^{<d-1}[X]$$

This basic property is central in the construction of the algorithm Eval, since it makes it possible to construct it simply as an IOP $\langle P,\,V\rangle$ designed to determine whether $x\mapsto\dfrac{f(x)-y}{x-z}\in\mathbb{F}^{<d-1}[X]$.

3 Construction of a PCS from an IOPP

This section presents a first construction of a PCS from an IOPP specified over a Reed-Solomon code. The construction of the underlying protocol will be studied in a later section, devoted to FRI, and will conclude this first approach. We begin by describing a few tools and properties that will be central to this construction.

3.1 Merkle Commitment Scheme

A fundamental step in our protocols relies on the ability of the Verifier to query the Prover about certain values of a committed oracle f over a domain $\mathcal{L}$, without assuming that the Prover is trustworthy. From this perspective, we introduce a simplified version of the notion of a *Merkle Commitment Scheme* (MCS).

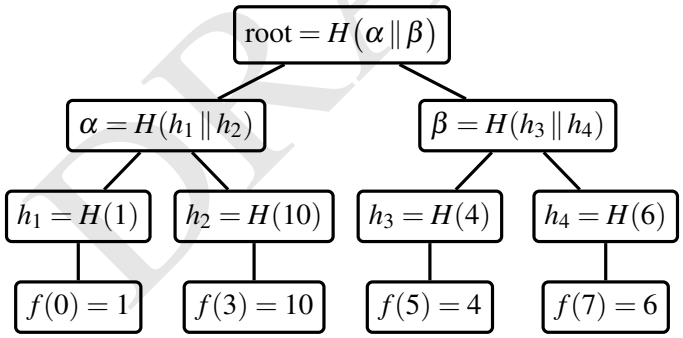
We assume that the domain $\mathcal{L}$ has cardinality a power of 2 (the construction can be generalized by *padding*, but we consider here the simplest case). The Prover then constructs the following tree and commits to its root.

Definition 3.1. Let $f : \mathcal{L} \longrightarrow \mathbb{F}$ be an oracle, and suppose that $\mathcal{L} = \{z_1, \ldots, z_n\}$ is ordered. The Merkle tree associated with f is the complete binary tree whose leaves are the hashes $H(f(z_i))$ and in which every internal node N is the hash of the concatenation of its two children N_{left} and N_{right}:

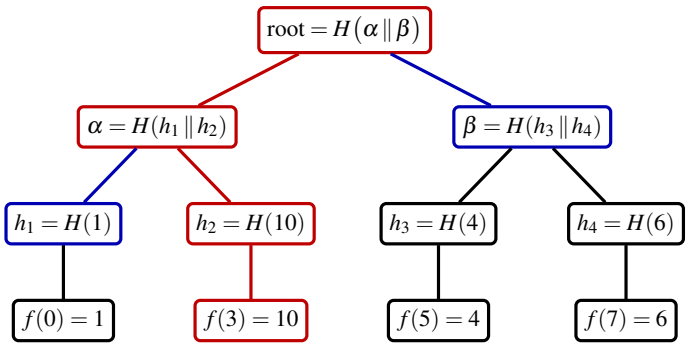
$$N = H\big(N_{\text{left}} \,\|\, N_{\text{right}}\big).$$

The root of this tree is called the **Merkle root** of the oracle f.

Example. Consider the oracle $f : \mathcal{L} \longrightarrow \mathbb{F}_{11}$ that associates to every $x \in \mathcal{L} = \{0,3,5,7\}$ the value x^2+1. We then construct the associated Merkle tree:



This construction, based on the security of the chosen hash function, makes it possible to convince the Verifier that the value provided by the Prover during a query indeed corresponds to the initially committed oracle. Indeed, to answer the query at the point z_i, the Prover sends $f(z_i)$, together with the opening path, composed of the auxiliary hashes needed to climb back up to the root. Returning to the previous example, the Prover sends $f(3)$, as well as $\mathsf{path} = (h_1, \beta)$, and the Verifier checks the following equalities:



$$h_2 = H(f(3))$$
$$\alpha = H(h_1 \,\|\, h_2)$$
$$\text{root} = H(\alpha \,\|\, \beta)$$

Under the assumption that the hash function used is collision-resistant, a Prover who wants to open the value at 3 consistently with the committed root must provide a path whose hashes recompute this root. Thus, after the root has been fixed, changing the opened value without changing the root would require finding a hash collision. Since this commitment to the root is made before the Verifier's queries, the root fixes the oracle values that can later be authenticated. The resource gain observed in this example remains unrepresentative because of the small size of the tree. In practice, this procedure is performed in $O(\log(n))$, where $n = |\mathcal{L}|$, whereas an explicit commitment to all the oracle values would cost $O(n)$.

A more complete and rigorous presentation of MCSs can be found in [CY24, section IV.18]. It provides in particular the fundamental properties of these schemes in the random oracle model:

1. an MCS satisfies a *binding* property;
2. an MCS satisfies an *extractability* property;
3. suitable randomized MCSs also satisfy a *hiding* property.

Binding guarantees that a single commitment cannot consistently authenticate two distinct oracles. Extractability guarantees that any response opened by the Prover is consistent with an oracle already fixed at the time of commitment. Hiding, when required, is obtained in the cited construction by adding fresh randomness to the leaf commitments; the simplified deterministic tree described above should therefore mainly be understood as an authentication mechanism. Thanks to these properties, one can describe protocols in a form where the Prover commits to an oracle, and then the Verifier accesses it only through a small number of queries, each response being accompanied by a verifiable opening relative to the initial commitment. In the following, for any oracle u, the short commitment of this oracle over a domain $\mathcal{L}$ sent by the Prover will be denoted as $[u]_{x\in\mathcal{L}}$ (or simply $[u]$) and can be thought of as a Merkle root.

The security analysis below is nevertheless carried out first in the oracle model. Thus, when a commitment C is fixed, we denote by $u_C \in \mathbb{F}^{\mathcal{L}}$ the oracle message associated with C in this ideal model. The Merkle root $[u_C]$ should be understood only as the later implementation of this oracle access through the BCS transformation; the proof below does not rely on reconstructing the whole oracle from a few Merkle openings.

3.2 Domain Extension for Eliminating Pretenders

The FRI protocol first appeared in [BSBHR18a] in 2018 and was quickly followed by the publication of an improved version [BSGKS19] in 2019. This new version introduces the *Domain Extension for Eliminating Pretenders* method, abbreviated as **DEEP**. First conceived as a path toward improving the soundness of FRI through the DEEP-FRI protocol, it was later reused in several contexts; in particular, it improves arithmetization through DEEP-ALI. We will see that the latter protocol enables the construction of SNARKs and STARKs without directly resorting to a PCS. Here, we present the method, the advantages it brings, as well as a first use in the construction of a PCS based on an IOPP.

Consider $\mathcal{L} \subseteq \mathbb{F}$, an oracle $f : \mathcal{L} \longrightarrow \mathbb{F}$, an m-tuple $\Omega = (z_1, \ldots, z_m)$ of pairwise distinct points in $\mathbb{F} \setminus \mathcal{L}$ and $\mathbf{y} = (y_1, \ldots, y_m) \in \mathbb{F}^m$. We then define

$$q = \mathrm{Quot}(f, \Omega, \mathbf{y}) : \mathcal{L} \longrightarrow \mathbb{F}$$

by

$$\forall x \in \mathcal{L}, \qquad q(x) = \frac{f(x) - I_\Omega(x)}{Z_\Omega(x)}$$

where

$$Z_\Omega(X) = \prod_{i=1}^{m} (X - z_i) \in \mathbb{F}[X]$$

and I_Ω denotes the unique polynomial of degree strictly less than m satisfying

$$\forall i \in \{1,\dots,m\}, \qquad I_\Omega(z_i) = y_i.$$

Lemma 3.1. *Let $\mathcal{C} = \mathrm{RS}[\mathbb{F},\mathcal{L},d]$, $\delta \in]0,1[$, $u \in \mathbb{F}^{\mathcal{L}}$, an integer $m < d$, an m-tuple $\Omega = (z_1,\dots,z_m)$ of pairwise distinct points in $\mathbb{F}\setminus\mathcal{L}$ and $\mathbf{y} = (y_1,\dots,y_m) \in \mathbb{F}^m$. For $q = \mathrm{Quot}(u,\Omega,\mathbf{y})$, the following assertions are equivalent:*

1. *There exists $Q \in \mathbb{F}^{<d-m}[X]$ such that $\Delta(q,Q) \le \delta$.*
2. *There exists $P \in \mathbb{F}^{<d}[X]$ such that $\Delta(u,P) \le \delta$ and $P(z_i) = y_i$ for every $i \in \{1,\dots,m\}$.*

Proof. If such a $Q \in \mathbb{F}^{<d-m}[X]$ exists, we set

$$P(X) = Q(X)Z_\Omega(X) + I_\Omega(X).$$

Since $Z_\Omega(x) \neq 0$ for every $x \in \mathcal{L}$, we have

$$\Delta(q,Q) = \Delta(q(x)Z_\Omega(x) + I_\Omega(x), Q(x)Z_\Omega(x) + I_\Omega(x)) = \Delta(u,P).$$

Moreover, we indeed have $\deg P < d$ and, for every $i \in \{1,\dots,m\}$,

$$P(z_i) = Q(z_i)Z_\Omega(z_i) + I_\Omega(z_i) = y_i.$$

Conversely, if such a $P \in \mathbb{F}^{<d}[X]$ exists, then the polynomial $P - I_\Omega$ vanishes at each of the z_i, hence Z_Ω divides $P - I_\Omega$. Thus,

$$Q = \frac{P - I_\Omega}{Z_\Omega}$$

is indeed a polynomial of degree strictly less than $d-m$. The same computations as those made above then give $\Delta(q,Q) \le \delta$. □

The interpretation of this lemma is the following: checking that u is δ-close to a polynomial P of the code $\mathrm{RS}[\mathbb{F},\mathcal{L},d]$ satisfying $P(z_i) = y_i$ for every $i \in \{1,\dots,m\}$ simply amounts to checking that $\mathrm{Quot}(u,\Omega,\mathbf{y})$ is δ-close to $\mathrm{RS}[\mathbb{F},\mathcal{L},d-m]$. In particular, we have

Corollary 3.2. *Let $m < d$, $u \in \mathrm{RS}[\mathbb{F},\mathcal{L},d]$ and $P \in \mathbb{F}^{<d}[X]$ be the polynomial that extends u over $\mathbb{F}$. If $P(z_i) = y_i$ for every $i \in \{1,\dots,m\}$, then $\mathrm{Quot}(u,\Omega,\mathbf{y}) \in \mathrm{RS}[\mathbb{F},\mathcal{L},d-m]$.*

First, computing $q(x)$ for $x \in \mathcal{L}$ and $q = \mathrm{Quot}(u,\Omega,\mathbf{y})$ requires no additional query compared with querying the Prover directly on $u(x)$. Thus, if the Verifier has an interactive protocol allowing him to be convinced that q is δ-close to $\mathrm{RS}[\mathbb{F},\mathcal{L},d-m]$, then he learns that u is δ-close to an oracle $f \in \mathrm{RS}[\mathbb{F},\mathcal{L},d]$ that extends over $\mathbb{F}$ with $f(z_i) = y_i$ for every $i \in \{1,\dots,m\}$. We therefore obtain an additional property with essentially the same amount of computation. To clarify the interest of this reduction, we begin by recalling the following very general lemma:

3.3 Study of the protocol

We now describe a first construction of a PCS from an IOPP. The protocol is presented in oracle form: the Prover first fixes the considered oracles, and then the Verifier accesses them only through point queries. In a later Merkle implementation, these oracle messages are replaced by commitments and authenticated openings through the BCS transformation. In the oracle-level proof below, the extractor is allowed to read the oracle u_C associated with an ideal commitment C; this is the idealization that Merkle binding and extractability later have to implement. In this presentation, we consider only evaluation queries at points located outside the domain $\mathcal{L}$, a necessary condition for the quotient studied above to be well defined:

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, d, m_{\max})$: the public parameters pp fix a domain $\mathcal{L}$ and a proximity radius δ. They must satisfy

$$0 < \delta \leq \frac{1-\rho}{2} \qquad \text{and} \qquad m_{\max} < d \qquad \text{with} \qquad 0 < \rho = \frac{d}{|\mathcal{L}|} < 1.$$

- $C \leftarrow \mathsf{Commit}(\mathsf{pp}, d, f(X))$: for $f \in \mathbb{F}^{<d}[X]$, one fixes the oracle $u_f \in \mathbb{F}^{\mathcal{L}}$ defined by $u_f(x) = f(x)$ for every $x \in \mathcal{L}$. In the oracle model, the commitment C is an ideal handle to this oracle; after Merkle compilation, it is implemented by the root $[u_f]_{x\in\mathcal{L}}$.

- $b \in \{0,1\} \leftarrow \mathsf{Open}(\mathsf{pp}, d, C, f(X))$: returns 1 if the oracle u_C associated with C is at relative distance at most δ from $f|_{\mathcal{L}}$, and 0 otherwise.

The algorithm Eval, for its part, relies on an IOPP associated with a Reed-Solomon code. The DEEP technique studied above, and in particular Lemma 3.1, allows it to be described through the quotient oracle associated with the oracle u_C:

- We fix $m \leq m_{\max}$, an m-tuple $\Omega = (z_1, \ldots, z_m)$ of pairwise distinct points in $\mathbb{F} \setminus \mathcal{L}$ and $\mathbf{y} = (y_1, \ldots, y_m)$, and then define the oracle

$$q_{u_C} = \mathrm{Quot}(u_C, \Omega, \mathbf{y}) \in \mathbb{F}^{\mathcal{L}}.$$

  We then set $\mathcal{C} = \mathrm{RS}[\mathbb{F}, \mathcal{L}, d-m]$ and run the protocol $\mathrm{IOPP}(q_{u_C}, \mathcal{C})$, which returns accept or reject. The output of Eval is defined by

$$b \in \{0,1\} \leftarrow \mathsf{Eval}(\mathsf{pp}, d, C, \Omega, \mathbf{y}; f(X))$$

  by setting $b = 1$ if $\mathrm{IOPP}(q_{u_C}, \mathcal{C}) \to \mathsf{accept}$, and $b = 0$ otherwise.

PROTOCOL 1 - PCS from an IOPP

Remark. The quotient oracle q_{u_C} should be understood here as a virtual oracle derived from the committed oracle u_C. In the oracle model, whenever the IOPP queries q_{u_C} at a point $x \in \mathcal{L}$, the Verifier queries $u_C(x)$ and computes

$$q_{u_C}(x) = \frac{u_C(x) - I_\Omega(x)}{Z_\Omega(x)}.$$

After Merkle compilation, this means that the Prover opens $u_C(x)$ with respect to the root representing C, while the other oracle messages required by the IOPP are authenticated by their own Merkle roots. Thus, the Prover is not allowed to choose an independent initial oracle for the proximity test: every queried value of the first IOPP oracle is checked against the value computed from the commitment C.

Lemma 3.3. *We denote by $\varepsilon = \varepsilon(\delta)$ the soundness error of the IOPP for the parameter δ. If no polynomial $f \in \mathbb{F}^{<d}[X]$ whose restriction to $\mathcal{L}$ is at relative distance at most δ from u_C satisfies $f(z_i) = y_i$ for every $i \in \{1, \ldots, m\}$, then, for every possibly malicious Prover $\tilde{P}$ in the IOPP instance with initial oracle q_{u_C},*

$$\mathbb{P}\left[\left\langle \tilde{P}(q_{u_C}, \mathcal{C}),\, V(\mathcal{C}) \right\rangle = \mathsf{accept}\right] \leq \varepsilon.$$

Proof. Suppose that no polynomial $f \in \mathbb{F}^{<d}[X]$ simultaneously satisfies $\Delta(u_C, f|_{\mathcal{L}}) \leq \delta$ and $f(z_i) = y_i$ for every $i \in \{1, \ldots, m\}$. If there existed a polynomial $Q \in \mathbb{F}^{<d-m}[X]$ such that $\Delta(q_{u_C}, Q) \leq \delta$, then Lemma 3.1 would provide a polynomial $f \in \mathbb{F}^{<d}[X]$ satisfying $\Delta(u_C, f|_{\mathcal{L}}) \leq \delta$ and $f(z_i) = y_i$ for every i, which would contradict the hypothesis. We therefore have

$$\Delta(q_{u_C}, \mathcal{C}) \geq \delta.$$

The soundness of the IOPP applied to the initial oracle q_{u_C} and to the code $\mathcal{C}$ then gives the announced bound. □

Theorem 3.4. *Under the oracle-commitment convention above, for every choice of $m \leq m_{\max}$, if the IOPP associated with the code $\mathcal{C} = \mathrm{RS}[\mathbb{F}, \mathcal{L}, d-m]$ has soundness error $\varepsilon(\delta)$, then the algorithms above define an oracle-level Polynomial Commitment Scheme for evaluation queries over $\mathbb{F} \setminus \mathcal{L}$.*

Proof. Let us verify the properties required of a PCS in this restricted oracle-level setting.
Completeness. If C is the ideal commitment to u_f with $u_f(x) = f(x)$ for every $x \in \mathcal{L}$, and if $f(z_i) = y_i$ for every $i \in \{1, \ldots, m\}$, then $u_C = u_f$ and Corollary 3.2 ensures that $q_{u_C} \in \mathcal{C}$. Completeness then follows directly from that of the IOPP.
Binding. Let $f_1, f_2 \in \mathbb{F}^{<d}[X]$ and let C be a commitment. If

$$\mathsf{Open}(\mathsf{pp}, d, C, f_1(X)) = 1 \qquad \text{and} \qquad \mathsf{Open}(\mathsf{pp}, d, C, f_2(X)) = 1,$$

then, by definition of Open,

$$\Delta(u_C, f_1|_{\mathcal{L}}) \leq \delta \qquad \text{and} \qquad \Delta(u_C, f_2|_{\mathcal{L}}) \leq \delta.$$

Since $\delta \leq \dfrac{1-\rho}{2}$, we are in the unique decoding regime for the code $\mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$, and there is therefore at most one polynomial of degree strictly less than d at relative distance at most δ from u_C. We deduce that $f_1 = f_2$, so the binding property holds with error 0 in this oracle-level model.
Knowledge extractability. Fix an admissible public instance $\mathsf{x} = (\mathsf{pp}, d, C, \Omega, \mathbf{y})$ and a malicious Prover $\tilde{P}$ for the protocol Eval. We denote

$$\rho' = \frac{d-m}{|\mathcal{L}|} \leq \rho.$$

Since $\delta \leq \dfrac{1-\rho}{2} \leq \dfrac{1-\rho'}{2}$, we remain within the unique decoding radius of the code

$$\mathcal{C} = \mathrm{RS}[\mathbb{F}, \mathcal{L}, d-m].$$

We then define an oracle-level polynomial-time extractor E which, on input x, reads the oracle u_C associated with the ideal commitment C, then computes

$$q_{u_C} = \mathrm{Quot}(u_C, \Omega, \mathbf{y}) \in \mathbb{F}^{\mathcal{L}},$$

and applies the Berlekamp-Welch algorithm to the word q_{u_C} with respect to the code $\mathcal{C} = \mathrm{RS}[\mathbb{F}, \mathcal{L}, d-m]$. If decoding fails, the extractor returns $\perp$. Otherwise, it obtains the unique polynomial $Q \in \mathbb{F}^{<d-m}[X]$ such that $\Delta(q_{u_C}, Q) \leq \delta$, then constructs

$$f(X) = Q(X) Z_\Omega(X) + I_\Omega(X) \in \mathbb{F}^{<d}[X]$$

and returns $f(X)$. The implication $(1) \Rightarrow (2)$ of Lemma 3.1 then shows that

$$\Delta(u_C, f|_{\mathcal{L}}) \leq \delta \qquad \text{and} \qquad f(z_i) = y_i \ \text{ for every } i \in \{1, \ldots, m\}.$$

By definition,

$$\mathsf{Open}(\mathsf{pp}, d, C, f(X)) = 1,$$

hence

$$(\mathsf{x}, f(X)) \in \mathcal{R}_{\mathsf{Eval}}.$$

Now suppose that the extractor returns $\perp$. By correctness of Berlekamp-Welch in the unique decoding regime, there is then no polynomial $Q \in \mathbb{F}^{<d-m}[X]$ such that $\Delta(q_{u_C}, Q) \leq \delta$. The converse implication $(2) \Rightarrow (1)$ of Lemma 3.1 therefore shows that there is no polynomial $f \in \mathbb{F}^{<d}[X]$ simultaneously satisfying

$\Delta(u_C, f|_{\mathcal{L}}) \leq \delta$ and $f(z_i) = y_i$ for every $i \in \{1, \ldots, m\}$. Hence there is no witness for the relation $\mathcal{R}_{\mathsf{Eval}}$. The preceding lemma then implies

$$\mathbb{P}\left[\left\langle \tilde{P}(\mathsf{x}),\, V(\mathsf{x})\right\rangle = \text{accept}\right] \leq \varepsilon.$$

Thus, in all cases,

$$\mathbb{P}\left[(\mathsf{x}, E^{\tilde{P}}(\mathsf{x})) \in \mathcal{R}_{\mathsf{Eval}}\right] \geq \mathbb{P}\left[\left\langle \tilde{P}(\mathsf{x}),\, V(\mathsf{x})\right\rangle = \text{accept}\right] - \varepsilon,$$

which shows that Eval has knowledge-soundness error ε for the relation $\mathcal{R}_{\mathsf{Eval}}$, and constitutes in particular an argument of knowledge for this relation in the oracle model. □

Remark. The construction above handles PCS evaluation claims only at points outside the domain $\mathcal{L}$, where the quotient oracle is well defined. Once the protocol has accepted, soundness guarantees, except with probability ε, that the committed oracle u_C is δ-close to a unique polynomial $P \in \mathbb{F}^{<d}[X]$ satisfying the claimed out-of-domain evaluations. In particular, u_C and P agree on at least $(1-\delta)|\mathcal{L}|$ points of $\mathcal{L}$.

Thus, after this PCS verification, one may still query the committed oracle at a point $z \in \mathcal{L}$ through an authenticated opening. This proves the value of $u_C(z)$. If z is sampled uniformly after the commitment, then this value coincides with $P(z)$ with probability at least $1-\delta$. However, for a fixed or adversarially chosen point $z \in \mathcal{L}$, such a query should not be interpreted as a PCS evaluation proof for $P(z)$, since z may belong to the exceptional set where u_C and P differ.

The PCS obtained above is functional, but it relies essentially on the assumption

$$\delta \leq \frac{1-\rho}{2},$$

that is, on the unique decoding regime of the underlying Reed-Solomon code. This assumption appears in the binding property, since the oracle u_C associated with a single ideal commitment C can then be δ-close to only one polynomial of degree strictly less than d, but also in knowledge extractability, where the Berlekamp-Welch algorithm is used to decode the quotient q_{u_C} uniquely in $\mathrm{RS}[\mathbb{F}, \mathcal{L}, d-m]$.
However, for optimization reasons, it is often desirable, especially for FRI, to consider proximity parameters larger than the unique decoding bound for our IOPPs. In this context, uniqueness generally disappears: a single word may admit several codewords at relative distance at most δ. Thus, the openings of a single commitment are no longer necessarily consistent with a unique polynomial, and the usual notion of a PCS becomes too rigid. Several solutions then exist: one of them consists in extending the notion of PCS and introducing *list polynomial commitment schemes*; this is in particular the approach chosen in [KPV19].
Another consists in dispensing with the notion of PCS and reducing our proofs of knowledge to the simple invocation of one or several IOPP instances. This solution then requires a specific arithmetization, described by *DEEP-ALI*. Instead of requiring that a commitment define a unique global polynomial, one only requires that it be close to a specific Reed-Solomon code. As explained in [Hab22], in the unique decoding regime this approach is equivalent to using a PCS in the classical sense, while offering the possibility of dispensing with this overly strict notion beyond this bound.

4 ARITHMETIZATION

The tools and protocols presented above play a fundamental role in the construction of proof systems of the SNARK and STARK type. They allow the Prover to convince the Verifier that he knows algebraic objects, typically polynomials, satisfying certain properties. It remains, however, to connect the execution of a program to such algebraic objects. This is precisely the role of arithmetization, which is the subject of this section. There are different approaches to this step, often chosen according to the proof system. The description in this section is inspired by the techniques presented in [MF23] and [BSGKS19].

In this approach, this arithmetization often relies on a dedicated virtual machine, called a *zkVM*, whose role is to execute the program while producing an execution trace. This trace therefore concerns not only the executed program, but also the entire internal machinery of the virtual machine. In this sense, one does not directly prove the correct execution of the program, but the correct execution of the whole procedure for executing the program in the zkVM. The generation of this trace and of the associated constraints is therefore complex and specific to each zkVM. For this reason, in this report we restrict ourselves to a simple illustrative example.

4.1 Illustrative construction of an AIR

The idea of an *Algebraic Intermediate Representation* (AIR) is to represent an execution by a *trace*, that is, an array of values, and then to impose *algebraic constraints* on this trace. In the setting of STARKs, these computations are carried out in a finite field $\mathbb{F}$.

To illustrate this idea, consider a program that computes the N-th term of the Fibonacci sequence, defined by $F_1 = F_2 = 1$ and $F_{n+2} = F_{n+1} + F_n$. At each step, the state of the computation is described by two registers, denoted a and b, which respectively contain the current term and the next term. Row i of the trace will therefore encode the pair $(a_i, b_i) = (F_{i+1}, F_{i+2})$.

Algorithm 1 Iterative computation of Fibonacci

Require: $N \geq 2$
1: $a \leftarrow 1$
2: $b \leftarrow 1$
3: **for** $i = 0$ **to** $N-2$ **do**
4: record the trace row $(a_i, b_i) = (a, b)$
5: $(a, b) \leftarrow (b, a+b)$
6: **end for**
7: record the last row $(a_{N-1}, b_{N-1}) = (a, b)$
8: **return** a

We thus obtain a two-column execution trace. For $N = 4$, it has the following form:

Row i	a_i	b_i
0	1	1
1	1	2
2	2	3
3	3	5

Table 1: Execution trace for the Fibonacci example.

The construction of the AIR then consists of translating the update $(a, b) \mapsto (b, a+b)$ into algebraic

constraints between two consecutive rows. For every $0 \leq i \leq N-2$, we impose

$$a_{i+1} - b_i = 0, \qquad b_{i+1} - (a_i + b_i) = 0.$$

To these transition constraints, we add boundary constraints in order to fix the initial state, as well as, possibly, a final constraint to impose the public output.

Type	Constraint	Interpretation
Initial boundary	$a_0 = 1$	fixes the first term of the sequence
Initial boundary	$b_0 = 1$	fixes the second term of the sequence
Transition	$a_{i+1} - b_i = 0$	the left register receives the previous value of the right one
Transition	$b_{i+1} - (a_i + b_i) = 0$	the right register contains the sum of the two previous terms
Final boundary (optional)	$a_{N-1} = y$	imposes that the public output is $y = F_N$

Table 2: AIR constraints for the Fibonacci example.

These relations are naturally reformulated in the form of *constraint polynomials*. If we denote by (x_1, x_2) the current row of the trace and by (y_1, y_2) the next row, then the transition constraints of the Fibonacci example are given by the two polynomials

$$P_1(x_1, x_2, y_1, y_2) = y_1 - x_2, \qquad P_2(x_1, x_2, y_1, y_2) = y_2 - (x_1 + x_2).$$

A valid trace then satisfies, for every pair of consecutive rows $\big((a_i, b_i), (a_{i+1}, b_{i+1})\big)$,

$$P_1(a_i, b_i, a_{i+1}, b_{i+1}) = 0, \qquad P_2(a_i, b_i, a_{i+1}, b_{i+1}) = 0.$$

Similarly, the initial conditions can be viewed as boundary polynomials:

$$B_1(x_1, x_2) = x_1 - 1, \qquad B_2(x_1, x_2) = x_2 - 1,$$

which must vanish on the first row of the trace. Finally, if one wants to impose a public output $y = F_N$, one may add the final polynomial, which must vanish on the last row:

$$B_f(x_1, x_2) = x_1 - y.$$

It is worth specifying that the trace is not indexed directly by integers, but by the elements of a cyclic multiplicative subgroup $G = \langle g \rangle \subset \mathbb{F}^*$, called the *trace domain*. This group has size $\geq N$ and allows the use of the FFT needed for efficient interpolation. In general, one chooses g as a 2^r-th root of unity, where r is the smallest integer such that $2^r \geq N$, and pads the trace: one may then assume $N = 2^r$ without loss of generality. Thus, the i-th row of the trace is associated with the point $g^i \in G$.

It then remains to interpolate each of our columns into polynomials of degree $< N$, called *trace polynomials*, representing each register. In the Fibonacci example, we have $A, B \in \mathbb{F}^{<N}[X]$, such that

$$A(g^i) = a_i, \qquad B(g^i) = b_i, \qquad 0 \leq i \leq N-1.$$

Passing to the next row is then expressed by the multiplicative shift $X \mapsto gX$. The transition constraints are reformulated as polynomial relations evaluated on the domain:

$$A(gX) - B(X) = 0, \qquad B(gX) - A(X) - B(X) = 0.$$

Remark. In this brief presentation, we restrict ourselves to traces with transition constraints of order 1, in the sense that these constraints involve only two successive steps. The articles [BSGKS19] and [MF23] use the notion of a *mask*, making it possible to generalize the AIR to higher-order transitions.

4.2 Formal definition of the AIR and APR relation

We now present the formal definition of the AIR for an arbitrary trace, following the approach of [Hab22]. We also define the associated *Algebraic Placement and Routing* relation, whose goal is the construction of a specific IOP whose acceptance is equivalent, except with small probability, to knowledge of valid trace polynomials. From the preceding illustrative presentation, an execution trace can be summarized by the following figure:

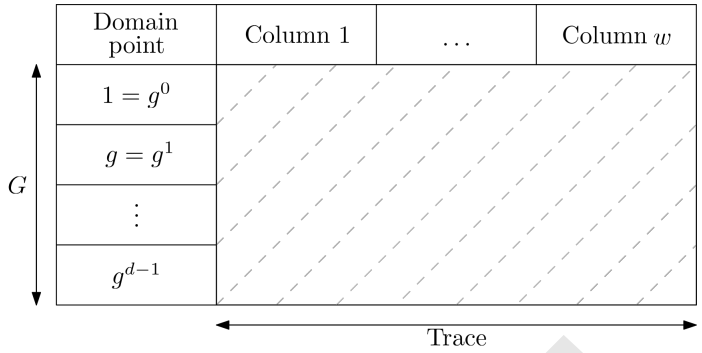


Figure 2: Trace representation

Definition 4.1 (AIR)**.** An AIR of size w defined on a trace domain $G \subset \mathbb{F}^*$ of order d consists of

- constraint polynomials $P_1, \ldots, P_c \in \mathbb{F}[X_1, \ldots, X_w, Y_1, \ldots, Y_w]$;
- cosets $H_1, \ldots, H_c$ of subgroups of G, contained in G: for every $i \in \{1, \ldots, c\}$, there exist $G_i \leq G$ and $a_i \in G$ such that $H_i = a_i \cdot G_i$. This coset is said to be non-trivial if $a_i \notin G_i$.

We say that a w-tuple $(f_1, \ldots, f_w)$ of polynomials in $\mathbb{F}^{<d}[X]$ satisfies the AIR if, for every $i \in \{1, \ldots, c\}$ and every $x \in H_i$, we have

$$P_i(f_1(x), \ldots, f_w(x), f_1(g \cdot x), \ldots, f_w(g \cdot x)) = 0$$

Moreover, we denote $t := \max_i \deg(P_i)$.

The notion of APR consists of reducing the AIR satisfiability problem to a membership problem for an RS code. First, observe that, for every valid w-tuple $(f_1, \ldots, f_w)$, one can define

$$\forall i \in \{1, \ldots, c\}, \qquad Q_i(X) := \frac{P_i(f_1(X), \ldots, f_w(X), f_1(g \cdot X), \ldots, f_w(g \cdot X))}{Z_{H_i}(X)} \in \mathbb{F}[X].$$

In this case, for every i we have

$$\deg(Q_i) \leq \deg(P_i)(d-1) - |H_i| < t \cdot d.$$

Thus, proving satisfaction of the AIR amounts to proving that the oracles derived from the Q_i belong to the code $\mathrm{RS}[\mathbb{F}, \mathcal{L}, t \cdot d]$.

Definition 4.2 (APR Relation)**.** We define the relation $\mathcal{R}_{\mathrm{APR}}$ from the following conditions:

1. **Instance format.** We set $\mathsf{x} = (\mathbb{F}, g, w, \mathsf{Cst})$ with
   - $\mathbb{F}$ a finite field;
   - g an element of the multiplicative group $\mathbb{F}^*$ of order d. We denote $G := \langle g \rangle$;
   - w a strictly positive integer;

- Cst a collection of $c \geq 1$ constraints, that is:
  (i) $(H_1, \ldots, H_c)$ cosets of subgroups of G;
  (ii) $(P_1, \cdots, P_c)$ polynomials in $\mathbb{F}[X_1, \ldots, X_w, Y_1, \ldots, Y_w]$.

2. **Witness format.** We set w to be the w-tuple $(f_1, \ldots, f_w)$ of polynomials in $\mathbb{F}[X]$.

3. **Relation.** We have $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}_{\text{APR}}$ if and only if:

   - for every $j \in \{1, \ldots, w\}$, we have $\deg(f_j) < d$;
   - for every $i \in \{1, \ldots, c\}$, the following quotient is a polynomial:

$$Q_i(X) := \frac{P_i(f_1(X), \ldots, f_w(X), f_1(g \cdot X), \ldots, f_w(g \cdot X))}{Z_{H_i}(X)} \in \mathbb{F}[X].$$

Remark. The coset structure imposed on the sets (H_i) in particular allows the efficient computation of the polynomials Z_{H_i}: for a multiplicative subgroup G_i of $\mathbb{F}^*$ of order d_i, we have $G_i = \{z \in \mathbb{F}^* \mid z^{d_i} - 1 = 0\}$. Thus, for $a_i \in \mathbb{F}^*$, we have

$$H_i := a_i \cdot G_i = \{\omega \in \mathbb{F}^* \mid \exists z \in \mathbb{F}^*,\ \omega = a_i \cdot z \text{ and } z^{d_i} - 1 = 0\} = \{\omega \in \mathbb{F}^*,\ \omega^{d_i} - a_i^{d_i} = 0\}$$

We then have $Z_{H_i}(X) = X^{d_i} - a_i^{d_i}$.

4.3 Algebraic Linking IOP

The *Algebraic Linking IOP* (ALI) consists of an IOP protocol for the relation $\mathcal{R}_{\text{APR}}$. Here, we are more particularly interested in *DEEP-ALI*, introduced in [BSGKS19], which is based on an IOPP and on the DEEP technique, and which recalls the construction of the PCS of protocol 1. The presentation below is notably inspired by the approach of [Hab22], where the different quotients of DEEP-ALI are grouped into a single IOPP instance. More precisely, we consider a particular IOPP, called *batched* and denoted b-IOPP, which considers not a single committed oracle, but a tuple of committed oracles. The construction of such a protocol, as well as the associated properties, are discussed in section 5.5.
For now, we simply mention that for a code $\mathcal{C} = \text{RS}[\mathbb{F}, \mathcal{L}, d]$ and a proximity parameter $\delta > 0$, if b-IOPP$_\delta(h_1, \ldots, h_m; \mathcal{C}) \to$ accept, then, except with probability $\varepsilon_{\text{IOPP}}$, there exists a subset $S \subset \mathcal{L}$ of density greater than $1 - \delta$ and polynomials $p_1, \ldots, p_m \in \mathbb{F}^{<d}[X]$ such that, for every $i \in \{1, \ldots, m\}$, we have $h_i|_S = p_i|_S$.

To link this relation to a single proximity test, we introduce the *selector polynomials*:

$$s_i(X) := \frac{Z_G(X)}{Z_{H_i}(X)} \in \mathbb{F}[X].$$

These allow the degrees to be harmonized in the verification of the constraints:

Lemma 4.1. *Let x be an APR instance. If $(\mathsf{x}, (f_1, \ldots, f_w)) \in \mathcal{R}_{\text{APR}}$, then, for every $i \in \{1, \ldots, c\}$, we have*

$$s_i(X) P_i(f_1(X), \ldots, f_w(X), f_1(g \cdot X), \ldots, f_w(g \cdot X)) \equiv 0 \mod Z_G.$$

Conversely, knowing polynomials $f_1, \ldots, f_w$ satisfying the congruences above makes it possible to extract a witness for $\mathcal{R}_{\text{APR}}$ in $O(wd^2)$.

Proof. If $(\mathsf{x}, (f_1, \ldots, f_w)) \in \mathcal{R}_{\text{APR}}$, then $P_i(f_1(X), \ldots, f_w(X), f_1(g \cdot X), \ldots, f_w(g \cdot X)) = Z_{H_i}(X) Q_i(X)$ for some $Q_i \in \mathbb{F}[X]$. Since $Z_G = s_i Z_{H_i}$, the announced congruence follows.

For the converse, it suffices to consider, for every $j \in \{1, \ldots, w\}$, the polynomial $\tilde{f}_j$ defined as the remainder in the Euclidean division of f_j by Z_G. We then observe that $\deg(Z_G) = d$, that $\deg(\tilde{f}_j) < d$, and that

$$P_i(f_1(X), \ldots, f_w(X), f_1(g \cdot X), \ldots, f_w(g \cdot X)) \equiv P_i(\tilde{f}_1(X), \ldots, \tilde{f}_w(X), \tilde{f}_1(g \cdot X), \ldots, \tilde{f}_w(g \cdot X)) \mod Z_G.$$

□

For a valid witness and $\lambda = (\lambda_1, \ldots, \lambda_c) \in \mathbb{F}^c$, if we set

$$g_\lambda(X) := \sum_{i=1}^{c} \lambda_i Q_i(X),$$

then $\deg(g_\lambda) < t \cdot d$ and $\sum_{i=1}^{c} \lambda_i s_i(X) P_i(f_1(X), \ldots, f_w(X), f_1(g \cdot X), \ldots, f_w(g \cdot X)) = Z_G(X) g_\lambda(X)$.
Finally, for every polynomial $h \in \mathbb{F}^{<t \cdot d}[X]$, we define $\mathsf{Split}(h, d) = (h_0, \ldots, h_{t-1})$ where the h_i are the unique polynomials in $\mathbb{F}^{<d}[X]$ such that

$$h = \sum_{\ell=0}^{t-1} h_\ell X^{\ell \cdot d}.$$

This decomposition, associated with a b-IOPP, makes it possible to reduce membership of an oracle in $\mathrm{RS}[\mathbb{F}, \mathcal{L}, t \cdot d]$ to a single b-IOPP instance with t oracles over $\mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$. In the extractability analysis, we need to consider the parameters $d_+ := d + 2$ and $\rho_+ = d_+ / |\mathcal{L}|$. These parameters make it possible to fix a context that takes into account the fact that the evaluation quotients have denominators of degree at most two, as in the DEEP-ALI analysis of [Sta23].
We now present the DEEP-ALI protocol associated with an instance $\mathsf{x} = (\mathbb{F}, g, w, \mathsf{Cst})$:

0. The Prover and the Verifier agree on a domain $\mathcal{L} \subset \mathbb{F}^*$ such that $g \cdot \mathcal{L} = \mathcal{L}$ and $|\mathcal{L}| = \rho^{-1} d$ for some $\rho \in]0,1[$, and on a proximity parameter $0 < \delta < 1 - \sqrt{\rho_+}$.

1. The Prover sends oracles $[f_1], \ldots, [f_w]$ over $\mathcal{L}$.

2. The Verifier sends $\lambda = (\lambda_1, \ldots, \lambda_c) \leftarrow\$\ \mathbb{F}^c$.

3. For every $i \in \{1, \ldots, c\}$, the Prover computes Q_i and then

$$g_\lambda = \sum_{i=1}^{c} \lambda_i Q_i.$$

   He computes $(g_{\lambda,0}, \ldots, g_{\lambda,t-1}) = \mathsf{Split}(g_\lambda, d)$ and sends the oracles $[g_{\lambda,0}], \ldots, [g_{\lambda,t-1}]$ over $\mathcal{L}$.

4. The Verifier sends $z \leftarrow\$\ \mathbb{F}^* \setminus (\mathcal{L} \cup G)$.

5. The Prover sends:

   (i) for every $j \in \{1, \ldots, w\}$, $\quad a_{j,0} := f_j(z) \quad$ and $\quad a_{j,1} := f_j(g \cdot z)$;

   (ii) for every $\ell \in \{0, \ldots, t-1\}$, $\quad b_\ell := g_{\lambda,\ell}(z)$.

6. The Verifier computes $b = \sum_{\ell=0}^{t-1} b_\ell z^{\ell \cdot d}$ and checks whether

$$\sum_{i=1}^{c} \lambda_i s_i(z) P_i(a_{1,0}, \ldots, a_{w,0}, a_{1,1}, \ldots, a_{w,1}) = Z_G(z) b.$$

   If the check fails, then the protocol returns reject and stops.

7. The Prover and the Verifier engage in a b-IOPP protocol on the following oracles:

$$\forall j \in \{1, \ldots, w\},\ q_j := \mathrm{Quot}(f_j, (z, g \cdot z), (a_{j,0}, a_{j,1})) \ ;$$

$$\forall \ell \in \{0, \ldots, t-1\},\ r_\ell := \mathrm{Quot}(g_{\lambda,\ell}, z, b_\ell).$$

   The protocol returns accept if accept $\leftarrow$ b-IOPP$_\delta(q_1, \ldots, q_w, r_0, \ldots, r_{t-1}; \mathcal{C})$ with $\mathcal{C} = \mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$, and returns reject otherwise.

PROTOCOL 2 - DEEP-ALI

Before studying the soundness of this IOP, let us begin with a few remarks:

1. In the usual FRI instantiation, $\mathcal{L}$ is chosen as a coset $a \cdot \tilde{G}$ of a multiplicative subgroup $\tilde{G} \leq \mathbb{F}^*$ containing G and satisfying $|\tilde{G}| = |\mathcal{L}| = \rho^{-1} d$. This situation automatically implies $g \cdot \mathcal{L} = \mathcal{L}$. In the presentation of FRI given later, we moreover place ourselves in the particular case $\rho^{-1} = 2^{\mathcal{R}}$.

2. In the final step, the oracles q_j and r_ℓ should be understood as oracles derived from the previous commitments. In other words, when a value at a point $x \in \mathcal{L}$ is required by the b-IOPP, the Prover opens the corresponding value of f_j or of $g_{\lambda,\ell}$ at x, and then the Verifier computes the announced quotient value directly.

[Complexity]

The soundness analysis detailed in [Hab22] splits this error into three distinct parts, which we describe informally:

- ε_b, the error arising from batching several oracles into a single IOPP instance;
- ε_d, the probability of drawing a $z \in \mathbb{F}^* \setminus (\mathcal{L} \cup G)$ favorable to a malicious Prover;
- $(\varepsilon_1, \ldots, \varepsilon_r)$, the round-by-round soundness of the IOPP used.

The value of ε_b will be discussed more precisely in section 5.5, and the value of ε_d comes essentially from Lemma A.1 (Schwartz-Zippel). This approach finally gives:

Theorem 4.2 (DEEP-ALI soundness)**.** *Consider the DEEP-ALI protocol 4.3 with the same notation and parameters, and let $L^+ = 1/(2\sqrt{\rho_+}(1-\sqrt{\rho_+}-\delta))$. If the IOPP has r rounds, then the round-by-round soundness of DEEP-ALI is given by $(\varepsilon_b, \varepsilon_d, \varepsilon_1, \ldots, \varepsilon_r)$ with:*

- $\varepsilon_b = L^+ \dfrac{1}{|\mathbb{F}|}$;
- $\varepsilon_d = L^+ \dfrac{t(d_+ - 1) + d - 1}{|\mathbb{F}^* \setminus (\mathcal{L} \cup G)|}$;
- $(\varepsilon_1, \ldots, \varepsilon_r)$ *the round-by-round soundness of the IOPP.*

In particular, a global soundness error is given by

$$\varepsilon_{\text{ALI}} = \varepsilon_b + \varepsilon_d + \varepsilon_{\text{IOPP}} \qquad \text{with} \quad \varepsilon_{\text{IOPP}} = \sum_{i=1}^{r} \varepsilon_i.$$

We now turn to knowledge extraction, which is necessary to ensure the construction of a SNARK through the BCS construction (see Theorem 2.7). To build an extractor, we use results and the method of [Sta23]. We begin by presenting the following result, based on a repetition of the Guruswami-Sudan algorithm:

Lemma 4.3 (Correlated agreement extractor)**.** *Let $d \in \mathbb{N}$, $\rho = d/|\mathcal{L}|$, $\delta \in]0, 1-\sqrt{\rho}[$, and $f_1, \ldots, f_w \in \mathbb{F}^{\mathcal{L}}$. There exists a probabilistic algorithm polynomial in $1/\rho$, $1/(1-\sqrt{\rho}-\delta)$, w, and $\log|\mathbb{F}|$ returning the list $(\mathcal{S}_1, \ldots, \mathcal{S}_L)$ of all maximal configurations $\mathcal{S}_v = (S_v, p_{v,1}, \ldots, p_{v,w})$ such that $S_v \subset \mathcal{L}$ has density at least $1-\delta$, $p_{v,j} \in \mathbb{F}^{<d}[X]$, and $f_j|_{S_v} = p_{v,j}|_{S_v}$ for every j. Moreover, the size of this list is bounded by $1/(2\sqrt{\rho}(1-\sqrt{\rho}-\delta))$.*

Proof. See [Sta23, section 5.5]. □

The analysis of [Hab22] then provides the local knowledge error

$$\mathsf{e}_{\text{loc}} := \varepsilon_{\text{ALI}} = L^+ \left(\frac{1}{|\mathbb{F}|} + \frac{t(d_+ - 1) + d - 1}{|\mathbb{F}^* \setminus (\mathcal{L} \cup G)|} \right) + \varepsilon_{\text{IOPP}}$$

for the following theorem:

Theorem 4.4 (Local extraction for DEEP-ALI)**.** *Consider the DEEP-ALI protocol 4.3, with the parameters above. Suppose that a deterministic Prover $\widetilde{P}$ sends as its first message the oracles*

$$[f_1], \ldots, [f_w]$$

and that, conditionally on this first message, the Verifier accepts with probability strictly greater than e_{loc}. Then there exists a local polynomial-time extractor E_{loc} which, from the oracles $f_1, \ldots, f_w$, returns a witness

$$(p_1, \ldots, p_w)$$

such that

$$(\mathsf{x}, (p_1, \ldots, p_w)) \in \mathcal{R}_{\text{APR}}.$$

Proof. The extractor reads the oracles $f_1, \dots, f_w$ over $\mathcal{L}$, then applies the correlated agreement decoder with degree d_+. It obtains a list $(\mathcal{S}_1, \dots, \mathcal{S}_L)$ of size at most L^+, consisting of maximal configurations. By the proof of Theorem 8 of [Hab22], applied to the fixed first message above, the hypothesis of acceptance strictly greater than $\mathrm{e}_{\mathrm{loc}}$ implies that at least one of these configurations $\mathcal{S} = (S, p_1, \dots, p_w)$ satisfies

$$\forall i \in \{1, \dots, c\}, \qquad s_i(X) P_i(p_1(X), \dots, p_w(X), p_1(gX), \dots, p_w(gX)) \equiv 0 \mod Z_G.$$

The extractor therefore tests all configurations in the list. When it finds one satisfying these congruences, it considers $\tilde{p}_j$, the remainder in the division of p_j by Z_G. Thus, by Lemma 4.1, we obtain that $(\tilde{p}_1, \dots, \tilde{p}_w)$ is a witness for $\mathcal{R}_{\mathrm{APR}}$. □

Corollary 4.5 (Global extractor)**.** *Assume that* $\mathrm{e}_{\mathrm{loc}} < 1/2$*. If a Prover* $\widetilde{P}$ *makes the Verifier accept with probability strictly greater than* $2\mathrm{e}_{\mathrm{loc}}$*, then there exists a global extractor* $E_{\mathrm{glob}}^{\widetilde{P}}$ *which returns a witness for* $\mathcal{R}_{\mathrm{APR}}$ *in expected polynomial time.*
In particular, DEEP-ALI is an argument of knowledge for $\mathrm{e} = 2\mathrm{e}_{\mathrm{loc}}$.

Proof. The global extractor repeats the following experiment:

1. it runs $\widetilde{P}$ until receiving the first message $([f_1], \dots, [f_w])$;
2. it then applies the local extractor E_{loc} to these oracles;
3. if a valid witness is obtained, it returns it; otherwise, it starts again.

Let A denote the event: "the first message obtained is good", meaning that the conditional acceptance probability of the Verifier after this first message is strictly greater than $\mathrm{e}_{\mathrm{loc}}$. If p denotes the total acceptance probability of $\widetilde{P}$, then

$$p \leq \mathbb{P}[A] + \mathrm{e}_{\mathrm{loc}}(1 - \mathbb{P}[A]).$$

Since $p > 2\mathrm{e}_{\mathrm{loc}}$, we obtain

$$\mathbb{P}[A] \geq \frac{p - \mathrm{e}_{\mathrm{loc}}}{1 - \mathrm{e}_{\mathrm{loc}}} > \frac{\mathrm{e}_{\mathrm{loc}}}{1 - \mathrm{e}_{\mathrm{loc}}} > \mathrm{e}_{\mathrm{loc}}.$$

Thus, the extractor encounters a good first message after an average number of iterations at most $1/\mathrm{e}_{\mathrm{loc}}$. The preceding theorem then ensures that E_{loc} extracts a valid witness. We therefore obtain a global extractor in expected polynomial time. □

Remark. Regarding the "argument of knowledge" aspect, we only consider the case where the acceptance probability of the malicious Prover is strictly greater than $2\mathrm{e}_{\mathrm{loc}}$. If this is not the case, the knowledge-error inequality is trivial, since

$$\mathbb{P}\left[(\mathsf{x}, E^{\widetilde{P}}(\mathsf{x})) \in \mathcal{R}_{\mathrm{APR}}\right] \geq 0 \geq \mathbb{P}\left[\langle \widetilde{P}(\mathsf{x}), V(\mathsf{x}) \rangle = \mathsf{accept}\right] - 2\mathrm{e}_{\mathrm{loc}}.$$

On instances outside the language, this bound reduces to classical soundness.

5 Fast Reed-Solomon Interactive Oracle Proofs of Proximity

The purpose of the previous sections was to show the relevance of IOPPs based on Reed-Solomon codes in the construction of SNARKs and STARKs. This section presents and studies the best-known IOPP, introduced in [BSBHR18a], namely the FRI protocol. Heuristically, this protocol relies on a technique called *folding*, which transforms the proximity problem for an RS code into a similar proximity problem, but over a smaller RS code.
This protocol has many variants intended to adapt it to various contexts, such as *BaseFold* in [ZCF23], *FRI-Binius* in [DP24], or *WHIR* in [ACFY24]. The study and analysis of the protocol are mainly based on the original paper [BSBHR18a], as well as on [Hab22] and [GMW25]. Finally, the security results stated here rely on results concerning Reed-Solomon codes, in particular on *Proximity Gaps*. This security is often formulated using [BSCI+20], but the recent results of [BSCH+25] improve some important bounds. The security analysis of FRI is based on these latter results.

5.1 Folding

An important technique in the construction of FRI is what is called *folding*. In general, a folding of order m consists of decomposing a polynomial $P \in \mathbb{F}^{<dm}[X]$ into m polynomials $P_1, \dots, P_m \in \mathbb{F}^{<d}[X]$, and then studying the polynomial

$$\mathsf{Fold}(P, \alpha) = P_1 + \alpha P_2 + \cdots + \alpha^{m-1} P_m.$$

Such a construction has very good properties with respect to Reed-Solomon codes and allows the degree of the polynomials to decrease geometrically. More general constructions, together with their properties, are presented in [ZCF23], [BSGKS19], and [GMW25]. In order to give a simple presentation of the FRI protocol, we restrict ourselves here to the study of the classical folding of order 2. When there is no ambiguity, we simply speak of folding.

Let us begin by briefly describing what our classical folding consists of. For an even integer d and $f \in \mathbb{F}^{<d}[X]$, we consider the polynomials $f_{\text{even}}, f_{\text{odd}} \in \mathbb{F}^{<d/2}[X]$, constructed respectively from the even-degree and odd-degree monomials of f.

Example. For $f = 2X^5 + 3X^4 - X^2 + 5X - 3$, we get

$$f_{\text{even}} = 3X^2 - X - 3 \qquad \text{and} \qquad f_{\text{odd}} = 2X^2 + 5.$$

This decomposition in particular satisfies

$$f(X) = f_{\text{even}}(X^2) + X f_{\text{odd}}(X^2),$$

and also

$$f_{\text{even}}(X^2) = \frac{f(X) + f(-X)}{2} \qquad \text{and} \qquad f_{\text{odd}}(X^2) = \frac{f(X) - f(-X)}{2X}.$$

The key points to note are the following:

- for every $z \in \mathbb{F}$, one can evaluate $f_{\text{even}}(z^2)$ and $f_{\text{odd}}(z^2)$ using only $f(z)$ and $f(-z)$.
- such a construction can be carried out for any oracle, and not only for polynomials. More precisely, for an oracle $u \in \mathbb{F}^{\mathcal{L}}$, an element $r \in \mathbb{F}$, and a point $a \in \mathcal{L}$, we define

$$\mathsf{Fold}(u, r)(a^2) := \frac{u(a) + u(-a)}{2} + r\frac{u(a) - u(-a)}{2a}.$$

The study of the new oracle obtained in this way is the starting point of the construction of FRI.

For our construction of FRI, it is necessary to consider a domain $\mathcal{L}_0$ in the form of a multiplicative coset of $\mathbb{F}^*$. That is, $\mathcal{L}_0 = aH$ for $a \in \mathbb{F}^*$ and H a multiplicative subgroup of $\mathbb{F}^*$. Finally, one must also ensure that $|\mathcal{L}_0| = n = 2^k$ for $k \in \mathbb{N}^*$. It is worth noting that all these conditions affect the choice of the field $\mathbb{F}$.

We define

$$\forall i \in \{1,\dots,k\}, \qquad \mathcal{L}_i = \{x^{2^i},\, x \in \mathcal{L}_0\}$$

and, for every $x \in \mathcal{L}_0$ and every $S \subseteq \mathcal{L}_0$,

$$\mathrm{Rt}(x) = \{w \in \mathcal{L}_0,\, w^2 = x\} \qquad \text{and} \qquad \mathrm{Rt}(S) = \bigsqcup_{x \in S} \mathrm{Rt}(x)$$

Remark. It is worth emphasizing that for every $i \in \{1,\dots,k\}$, we have $\mathcal{L}_i = \mathcal{L}_{i-1}^2$ and that, for any $i \in \{1,\dots,k\}$ and for every $x \in \mathcal{L}_i$, there exists $w \in \mathcal{L}_{i-1}$ such that $\mathrm{Rt}(x) = \{w, -w\}$.

As mentioned above, the main advantage of the folding technique comes from its connection with Reed-Solomon codes. More precisely, it involves a combination of oracles of the form $u_0 + r \cdot u_1$, as well as the property known as correlated agreement. This property, first named in [BSCI+20], is one of the main avenues for improving the soundness of FRI. It was recently improved in [BSCH+25, Theorem 4.6], and our security analysis is based on this version:

Theorem 5.1 (Correlated agreement under the Johnson bound)**.** *Let $\mathcal{C} = RS[\mathbb{F}, \mathcal{L}, d]$ have rate ρ, and let $n = |\mathcal{L}|$. Let $u_0, u_1 \in \mathbb{F}^{\mathcal{L}}$, $r \in \mathbb{F}$, $\delta \in]0, 1 - \sqrt{\rho}[$, and let $\varepsilon_{ca}(\mathcal{C}, \delta)$ be defined by*

$$\varepsilon_{ca}(\mathcal{C}, \delta) = \frac{1}{|\mathbb{F}|}\left(\frac{2\eta^5 + 3\eta\delta\rho}{3\rho^{3/2}} n + \frac{\eta}{\sqrt{\rho}}\right)$$

with $\eta = \max\left(\left\lceil \frac{\sqrt{\rho}}{1 - \sqrt{\rho} - \delta} \right\rceil, 3\right) + \frac{1}{2}$. *Then*

$$\mathop{\mathbb{P}}_{r}\left[\exists S \subseteq \mathcal{L} \text{ such that } \begin{smallmatrix} |S| \geq (1-\delta)\cdot|\mathcal{L}| \\ \exists v \in \mathcal{C},\ (u_0 + r\cdot u_1)|_S = v|_S \\ \exists i \in \{0,1\},\ \forall v \in \mathcal{C},\ u_i|_S \neq v|_S \end{smallmatrix}\right] \leq \varepsilon_{ca}(\mathcal{C}, \delta).$$

Remark. There are many formulations of the correlated agreement property, which are more or less equivalent. The choice made here is admittedly not the most natural one for a first encounter with the phenomenon (one may instead consult [BSCH+25, Theorem 4.2]), but it is particularly well suited to the proof given by [GMW25] for the soundness of FRI.

The following corollary, which is fundamental in the analysis of FRI, then follows from this theorem:

Corollary 5.2. *Let $0 < \delta < 1 - \sqrt{\rho}$, where ρ is the rate of the code $\mathcal{C}_0 = RS[\mathbb{F}, \mathcal{L}_0, d]$. Let $u \in \mathbb{F}^{\mathcal{L}_0}$ and $\mathcal{C}_1 = RS[\mathbb{F}, \mathcal{L}_1, d/2]$. Then*

1. *if $u \in \mathcal{C}_0$, then for every $r \in \mathbb{F}$, we have $\mathsf{Fold}(u, r) \in \mathcal{C}_1$;*
2. *we have*

$$\mathop{\mathbb{P}}_{r}\left[\exists S \subseteq \mathcal{L}_0 \text{ such that } \begin{smallmatrix} |S^2| \geq (1-\delta)\cdot|\mathcal{L}_1| \\ \exists v \in \mathcal{C}_1,\ \mathsf{Fold}(u,r)|_{S^2} = v|_{S^2} \\ \forall v \in \mathcal{C}_0,\ u|_S \neq v|_S \end{smallmatrix}\right] \leq \varepsilon_{ca}(\mathcal{C}_1, \delta).$$

In particular, if $\Delta(u, \mathcal{C}_0) > \delta$, then

$$\mathop{\mathbb{P}}_{r}[\Delta(\mathsf{Fold}(u,r), \mathcal{C}_1) \leq \delta] \leq \varepsilon_{ca}(\mathcal{C}_1, \delta),$$

Proof. The first point is immediate from the construction of the folding.
To prove the second point, it suffices to show that the event described in this corollary is included in the event of Theorem 5.1 applied to the code $\mathcal{C}_1$, which has the same rate ρ as $\mathcal{C}_0$. In other words, suppose that there exists a subset $S \subseteq \mathcal{L}_0$ such that

(i) $|S^2| \geq (1-\delta)|\mathcal{L}_1|$;

(ii) there exists $v \in \mathcal{C}_1$ such that $\mathsf{Fold}(u,r)|_{S^2} = v|_{S^2}$;

(iii) for every $v \in \mathcal{C}_0$, we have $u|_S \neq v|_S$.

Writing $\mathsf{Fold}(u,r) = u_0 + ru_1$ with the well-defined maps $u_0, u_1 \in \mathbb{F}^{\mathcal{L}_1}$, for every $a \in \mathcal{L}_0$,

$$u_0(a^2) = \frac{u(a)+u(-a)}{2} \qquad \text{and} \qquad u_1(a^2) = \frac{u(a)-u(-a)}{2a},$$

it then suffices to show that $S' := S^2$ satisfies

(i) $|S'| \geq (1-\delta)|\mathcal{L}_1|$;

(ii) there exists $v \in \mathcal{C}_1$ such that $(u_0 + r \cdot u_1)|_{S'} = v|_{S'}$;

(iii) there exists $i \in \{0,1\}$ such that, for every $v \in \mathcal{C}_1$, we have $u_i|_{S'} \neq v|_{S'}$.

Points (i) and (ii) are clearly satisfied. We prove point (iii) by contraposition: suppose that there exist $v_0, v_1 \in \mathcal{C}_1$ which respectively agree with u_0 and u_1 on S^2. Let $V_0, V_1 \in \mathbb{F}^{<d/2}[X]$ be the polynomials whose evaluations on $\mathcal{L}_1$ are v_0 and v_1, and set

$$V(X) = V_0(X^2) + XV_1(X^2).$$

Then

$$\deg(V) \leq \max\big(2\deg(V_0), 2\deg(V_1)+1\big) \leq d-1 < d.$$

Let $v \in \mathcal{C}_0$ be the codeword obtained by evaluating V on $\mathcal{L}_0$. For every $a \in \mathrm{Rt}(S^2)$, we have

$$v(a) = V_0(a^2) + aV_1(a^2) = v_0(a^2) + av_1(a^2) = u_0(a^2) + au_1(a^2) = u(a),$$

so v agrees with u on $\mathrm{Rt}(S^2) \supseteq S$. This contradicts (iii), which proves point (iii). Finally, by Theorem 5.1, we have

$$\mathop{\mathbb{P}}_{r}\left[\exists S \subseteq \mathcal{L}_0 \text{ such that } \begin{smallmatrix} |S^2| \geq (1-\delta)\cdot|\mathcal{L}_1| \\ \exists v \in \mathcal{C}_1,\ \mathsf{Fold}(u,r)|_{S^2} = v|_{S^2} \\ \forall v \in \mathcal{C}_0,\ u|_S \neq v|_S \end{smallmatrix}\right] \leq \mathop{\mathbb{P}}_{r}\left[\exists S' \subseteq \mathcal{L}_1 \text{ such that } \begin{smallmatrix} |S'| \geq (1-\delta)\cdot|\mathcal{L}_1| \\ \exists v \in \mathcal{C}_1,\ (u_0 + r\cdot u_1)|_{S'} = v|_{S'} \\ \exists i \in \{0,1\},\ \forall v \in \mathcal{C}_1,\ u_i|_{S'} \neq v|_{S'} \end{smallmatrix}\right] \leq \varepsilon_{ca}(\mathcal{C}_1, \delta).$$

Finally, suppose that $\Delta(u, \mathcal{C}_0) > \delta$ and that $\Delta(\mathsf{Fold}(u,r), \mathcal{C}_1) \leq \delta$. Then there exists $S' \subseteq \mathcal{L}_1$ such that $|S'| \geq (1-\delta)|\mathcal{L}_1|$ and a word $v \in \mathcal{C}_1$ satisfying $\mathsf{Fold}(u,r)|_{S'} = v|_{S'}$. Setting $S = \mathrm{Rt}(S')$, we have $S^2 = S'$ and

$$|S| = 2|S'| \geq (1-\delta)|\mathcal{L}_0|.$$

Thus no word of $\mathcal{C}_0$ can agree with u on S, otherwise we would have $\Delta(u, \mathcal{C}_0) \leq \delta$. The event $\Delta(\mathsf{Fold}(u,r), \mathcal{C}_1) \leq \delta$ is therefore included in the event studied above, which gives the last announced inequality. □

Remark. The proof presented above is independent of the value of ε_{ca}. Thus, for any ε satisfying the correlated agreement property, the previous corollary holds. In this sense, the improvement of the bound in Theorem 5.1 propagates to the whole soundness analysis of FRI.

5.2 Protocol Presentation

As in the PCS description given above, the protocol is presented in oracle form, which is reasonable in view of the construction of Merkle commitments. We fix a domain $\mathcal{L}_0$ as a coset of a multiplicative subgroup of $\mathbb{F}^*$ of order $n = 2^k$, an oracle $f_0 : \mathcal{L}_0 \longrightarrow \mathbb{F}$, and a rate $\rho = 2^{-\mathcal{R}}$, where $\mathcal{R} < k$ is a nonnegative integer. The FRI protocol then consists of two phases, COMMIT and QUERY, whose purpose is to convince the Verifier that f_0 is close to $\mathcal{C}_0 = \mathrm{RS}[\mathbb{F}, \mathcal{L}_0, d]$ with $d = \rho|\mathcal{L}_0| = 2^{k-\mathcal{R}}$.
We now fix an integer r such that $1 \leq r \leq k - \mathcal{R}$. The COMMIT phase then has r steps and allows the Prover to provide the Verifier with a sequence of oracle commitments. For every $i \in \{1, \ldots, r\}$, we set $\mathcal{C}_i = \mathrm{RS}[\mathbb{F}, \mathcal{L}_i, d/2^i]$.
The QUERY phase has $t \geq 1$ steps whose purpose is to convince the Verifier that the oracles committed by the Prover have indeed been obtained by successive foldings and that the final oracle is a word of the code $\mathcal{C}_r$. In practice, this last verification remains easy provided that r is chosen so that the final domain $\mathcal{L}_r$ has small size.
$\mathcal{L}_0$, ρ, and t are the parameters of FRI, and their choice affects the security of the protocol.

We now describe the two phases of the FRI protocol:

Phase COMMIT:

1. The Prover sends the oracle $[f_0(x)]_{x \in \mathcal{L}_0}$.
2. For every round $i = 1, \ldots, r$:
   (i) The Verifier sends a challenge $\alpha_i \in \mathbb{F}$.
   (ii) The Prover returns an oracle $[f_i(x)]_{x \in \mathcal{L}_i}$ with $f_i = \mathsf{Fold}(f_{i-1}, \alpha_i)$.

Phase QUERY:

3. The Verifier chooses $s_1, \ldots, s_t \in \mathcal{L}_0$ and sends $\alpha_{r+1} = (s_1, \ldots, s_t)$ to the Prover.
4. The Verifier reads f_r entirely on $\mathcal{L}_r$ and checks that $f_r \in \mathcal{C}_r$.
5. For every $j = 1, \ldots, t$:
   - $s \leftarrow s_j$.
   - For every $i = 1, \ldots, r$:
     (i) The Verifier queries f_{i-1} at the values $s^{2^{i-1}}$ and $-s^{2^{i-1}}$, then computes

$$\mathsf{Fold}(f_{i-1}, \alpha_i)(s^{2^i}) = \frac{f_{i-1}(s^{2^{i-1}}) + f_{i-1}(-s^{2^{i-1}})}{2} + \alpha_i \frac{f_{i-1}(s^{2^{i-1}}) - f_{i-1}(-s^{2^{i-1}})}{2s^{2^{i-1}}}.$$

     (ii) The Verifier queries f_i at the value s^{2^i} and checks that

$$\mathsf{Fold}(f_{i-1}, \alpha_i)(s^{2^i}) = f_i(s^{2^i}).$$

PROTOCOL 3 - FRI

Before studying the security of the protocol, it is useful to make a few remarks on how it runs in practice. These remarks concern the number of oracle queries made by the Verifier during the QUERY phase and appear, for example, in Dan Boneh's course [SB25b]. The optimization concerns the representation of the domains $\mathcal{L}_i$ and the construction of the associated Merkle trees, and reduces this

number of queries.

To simplify the presentation of this technique, we assume that $\mathcal{L}_0$ is a multiplicative subgroup of $\mathbb{F}^*$ generated by w. Thus, the naive representation of $\mathcal{L}_0$ is $[1, w, w^2, \ldots, w^{2^k-1}]$. The formal justification of the optimization described below is deferred to the appendix.
To describe a better representation, we define, for every $k > 0$ and $i \in \{0, \ldots, 2^k - 1\}$:

$$\mathrm{Renv}_k(i) = \sum_{j=0}^{k-1} a_j 2^{k-j-1} \quad \text{with } i = \sum_{j=0}^{k-1} a_j 2^j$$

with the convention $\mathrm{Renv}_0(0) = 0$. In other words, $\mathrm{Renv}_k(i)$ is the integer in $\{0, \ldots, 2^k - 1\}$ whose binary expansion is that of i read backwards. We then consider the numbering $[w^{\mathrm{Renv}_k(i)},\ i \in \{0, \ldots, 2^k - 1\}]$ and construct the complete binary tree whose leaves are the points of $\mathcal{L}_0$ for this representation. In this case, Proposition C.1, proved in the appendix, ensures that the values of sibling nodes and sibling leaves have the same square, this square being the value of the parent node.

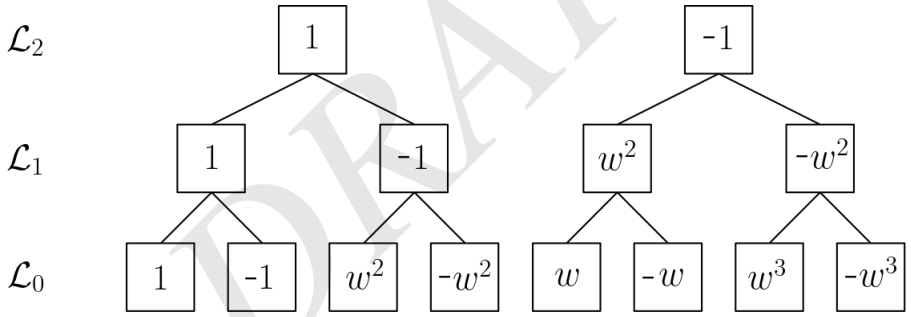
Example. For $k = 3$, we have $\mathcal{L} = \{1, w, \ldots, w^7\}$, which is represented in the reversed numbering as

$$1,\ w^4,\ w^2,\ w^6,\ w,\ w^5,\ w^3,\ w^7$$

which can also be rewritten as

$$1,\ -1,\ w^2,\ -w^2,\ w,\ -w,\ w^3,\ -w^3$$

We can thus construct the following binary tree:



Remark. Even though its description is given here for a multiplicative subgroup of $\mathbb{F}^*$, it is worth noting that this construction remains compatible with the cosets used in FRI.

The usefulness of such a construction comes from the fact that it allows the $\mathcal{L}_i$ to be described easily in a way adapted to Merkle commitments. Indeed, this construction makes it possible to check that, for an oracle f_i on $\mathcal{L}_i$ and for every $s \in \mathcal{L}_i$, the leaves corresponding to the values $f_i(s)$ and $f_i(-s)$ are siblings in the associated Merkle tree. The opening data for sibling leaves in a Merkle tree are exactly the same except for the leaves themselves. Thus, opening the values $f_i(s)$ and $f_i(-s)$ essentially requires only one query instead of two.
Finally, since this type of query appears at each step of the QUERY phase, such a preliminary construction saves about $r \cdot t$ queries.

5.3 Security Analysis

In this section, we verify that the FRI protocol presented above satisfies the properties specific to an IOPP. First, the first point of Corollary 5.2 ensures the completeness of FRI. For the study of soundness, we largely follow the proof given in [GMW25], itself written from the most recent results on *Proximity Gaps* obtained in [BSCH+25].

Theorem 5.3 (Soundness of FRI under the Johnson bound)**.** *For the construction of the FRI protocol 5.2 with r COMMIT rounds and t QUERY points, and for $0 < \delta < 1 - \sqrt{\rho}$, we have round-by-round soundness $(\varepsilon_{C,1}, \ldots, \varepsilon_{C,r}, \varepsilon_Q)$ with:*

- $\forall i \in \{1, \ldots, r\},\ \varepsilon_{C,i} \leq \varepsilon_{ca}(\mathcal{C}_i, \delta)$;
- $\varepsilon_Q < (1-\delta)^t$.

Before actually beginning the proof, let us define the state function State used in the study of round-by-round soundness. It is worth noting that it depends on the parameter δ chosen before the protocol. It is defined as follows:

1. **Empty transcript:** $\mathsf{State}(f_0, \emptyset) = 1 \iff f_0 \in \mathcal{C}_0$.
2. **Verifier interaction:** At round $i \in \{1, \ldots, r\}$ of COMMIT, the transcript has the form
$$\mathsf{tr} = \begin{cases} \emptyset & \text{if } i = 1 \\ (\alpha_1, f_1, \ldots, \alpha_{i-1}, f_{i-1}) & \text{otherwise.} \end{cases}$$
and the Verifier sends its message $\alpha_i \leftarrow \mathbb{F}$. If $\mathsf{State}(f_0, \mathsf{tr}) = 0$, then we set $\mathsf{State}(f_0, \mathsf{tr} \,\|\, \alpha_i) = 1$ if and only if there exists a subset $S \subseteq \mathcal{L}_{i-1}$ satisfying:
   (i) $|S^2| \geq (1-\delta)|\mathcal{L}_i|$;
   (ii) there exists $u' \in \mathcal{C}_i$ such that $\mathsf{Fold}(f_{i-1}, \alpha_i)|_{S^2} = u'|_{S^2}$;
   (iii) for every $u \in \mathcal{C}_{i-1}$, there exists $x \in S$ such that $u(x) \neq f_{i-1}(x)$.
3. **Complete transcript:** During the QUERY round, the transcript is then $\mathsf{tr} = (\alpha_1, f_1, \ldots, \alpha_r, f_r)$ and the Verifier sends $\alpha_{r+1} = (s_1, \ldots, s_t) \in \mathcal{L}_0^t$. If $\mathsf{State}(f_0, \mathsf{tr}) = 0$, then we set $\mathsf{State}(f_0, \mathsf{tr} \,\|\, \alpha_{r+1}) = 1$ if and only if we have
   (i) $f_r \in \mathcal{C}_r$;
   (ii) for every $j \in \{1, \ldots, t\}$ and every $i \in \{1, \ldots, r\}$, we have $\mathsf{Fold}(f_{i-1}, \alpha_i)(s_j^{2^i}) = f_i(s_j^{2^i})$.

Finally, we impose on State the properties of **Prover Interaction** and **Default Behavior**. The description above then fixes the two other conditions, so that State is indeed a state function.

We now want to use State to prove the soundness of FRI. To do so, we first introduce a few preliminary results. We consider $f_0 \in \mathbb{F}^{\mathcal{L}_0}$, $\mathsf{tr} = (\alpha_1, f_1, \ldots, \alpha_r, f_r)$, and set

$$\mathcal{A} = \left\{ z \in \mathcal{L}_0 \mid \forall i \in \{1, \ldots, r\},\ \mathsf{Fold}(f_{i-1}, \alpha_i)(z^{2^i}) = f_i(z^{2^i}) \right\}.$$

This is the set of points of $\mathcal{L}_0$ that are valid for the QUERY phase. We then define

$$\begin{cases} W_0 := \mathcal{A} \\ W_i := W_{i-1}^2 & \text{for } i \in \{1, \ldots, r\} \end{cases}$$

Finally, we suppose that $\Delta(f_0, \mathcal{C}_0) > \delta$ and that $\mathsf{State}(f_0, \mathsf{tr}) = 0$. We then have the following lemmas:

Lemma 5.4. *Suppose that for every $j \in \{0, \ldots, r\}$, we have $|W_j| \geq (1-\delta)|\mathcal{L}_j|$. Fix $i \in \{1, \ldots, r\}$. If there exists $u_i \in \mathcal{C}_i$ such that $u_i|_{W_i} = f_i|_{W_i}$, then there exists $u_{i-1} \in \mathcal{C}_{i-1}$ such that $u_{i-1}|_{W_{i-1}} = f_{i-1}|_{W_{i-1}}$. In particular, if $f_r \in \mathcal{C}_r$, then there exists $u_0 \in \mathcal{C}_0$ such that $u_0|_{W_0} = f_0|_{W_0}$.*

Proof. Suppose for contradiction that for every $u \in \mathcal{C}_{i-1}$, we have $u|_{W_{i-1}} \neq f_{i-1}|_{W_{i-1}}$. Set

$$\mathsf{tr}_{i-1} = \begin{cases} \emptyset & \text{if } i = 1 \\ (\alpha_1, f_1, \ldots, \alpha_{i-1}, f_{i-1}) & \text{otherwise.} \end{cases}$$

Then, with $S = W_{i-1}$, the three conditions in the definition of $\mathsf{State}(f_0, \mathsf{tr}_{i-1} \,\|\, \alpha_i) = 1$ are satisfied:

(i) we have $S^2 = W_i$, hence $|S^2| = |W_i| \geq (1-\delta)|\mathcal{L}_i|$;

(ii) by hypothesis, there exists $u_i \in \mathcal{C}_i$ such that $u_i|_{W_i} = f_i|_{W_i}$, and for every $x \in W_i$, there exists $z \in \mathcal{A}$ such that $x = z^{2^i}$. Thus, by definition of $\mathcal{A}$, we have

$$\mathsf{Fold}(f_{i-1}, \alpha_i)(x) = \mathsf{Fold}(f_{i-1}, \alpha_i)(z^{2^i}) = f_i(z^{2^i}) = f_i(x).$$

We finally obtain $f_i|_{W_i} = \mathsf{Fold}(f_{i-1}, \alpha_i)|_{W_i}$.

(iii) this is precisely the hypothesis stated above.

Therefore $\mathsf{State}(f_0, \mathsf{tr}_{i-1} \,\|\, \alpha_i) = 1$. By default behavior, every extension of this transcript is still mapped to 1, in particular the complete transcript tr, which contradicts the hypothesis $\mathsf{State}(f_0, \mathsf{tr}) = 0$. The particular assertion is obtained by iterating this implication for $i = r, r-1, \ldots, 1$ starting from $u_r = f_r$. □

Proof (of Theorem 5.3). We now resume the proof of the soundness of FRI:
COMMIT soundness. By construction of the state function and by Corollary 5.2, we have directly

$$\mathop{\mathbb{P}}_{\alpha_i}[\mathsf{State}(f_0, \mathsf{tr} \,\|\, \alpha_i) = 1 \mid \mathsf{State}(f_0, \mathsf{tr}) = 0] \leq \varepsilon_{ca}(\mathcal{C}_i, \delta).$$

QUERY soundness. We now use the lemmas introduced above. If $f_r \notin \mathcal{C}_r$, then for every α_{r+1} we immediately have $\mathsf{State}(f_0, \mathsf{tr} \,\|\, \alpha_{r+1}) = 0$. Hence suppose from now on that $f_r \in \mathcal{C}_r$. Consider the following two cases:

- Suppose that there exists $i \in \{1, \ldots, r\}$ such that $|W_i| < (1-\delta)|\mathcal{L}_i|$. By construction, $|W_{i-1}| \leq 2|W_i|$ and $|\mathcal{L}_{i-1}| = 2|\mathcal{L}_i|$, and therefore

$$\frac{|W_{i-1}|}{|\mathcal{L}_{i-1}|} \leq \frac{|W_i|}{|\mathcal{L}_i|} < 1 - \delta.$$

  Thus,

$$\frac{|\mathcal{A}|}{|\mathcal{L}_0|} = \frac{|W_0|}{|\mathcal{L}_0|} \leq \frac{|W_i|}{|\mathcal{L}_i|} < 1 - \delta.$$

- Otherwise, for every $i \in \{0, \ldots, r\}$, we have $|W_i| \geq (1-\delta)|\mathcal{L}_i|$, so by Lemma 5.4 applied to $i = r$, there exists $u_0 \in \mathcal{C}_0$ such that

$$u_0|_{W_0} = f_0|_{W_0}.$$

  Hence

$$\frac{|W_0|}{|\mathcal{L}_0|} \leq 1 - \Delta(u_0, f_0) < 1 - \delta$$

Since $W_0 = \mathcal{A}$, we deduce in all cases that

$$\frac{|\mathcal{A}|}{|\mathcal{L}_0|} < 1 - \delta.$$

Thus, for a uniform point $s \in \mathcal{L}_0$, we have

$$\mathop{\mathbb{P}}_{s}[s \in \mathcal{A}] = \frac{|\mathcal{A}|}{|\mathcal{L}_0|} < 1 - \delta.$$

Since $\alpha_{r+1} = (s_1, \ldots, s_t)$ is sampled uniformly in $\mathcal{L}_0^t$, the samples are independent, and acceptance of the QUERY phase requires all the s_j to belong to $\mathcal{A}$. We therefore obtain:

$$\mathop{\mathbb{P}}_{\alpha_{r+1}}[\mathsf{State}(f_0, \mathsf{tr} \,\|\, \alpha_{r+1}) = 1] \leq \mathop{\mathbb{P}}_{(s_1, \ldots, s_t)}[\forall j \in \{1, \ldots, t\},\ s_j \in \mathcal{A}] < (1-\delta)^t,$$

which proves that $\varepsilon_Q < (1-\delta)^t$. □

Corollary 5.5. *The global soundness error of the FRI protocol with the same parameters as above is then*

$$\varepsilon_{FRI} \leq \sum_{i=1}^{r} \varepsilon_{C,i} + \varepsilon_Q.$$

We set in particular

$$\varepsilon_C = \sum_{i=1}^{r} \varepsilon_{C,i}$$

and verify that

$$\varepsilon_C \leq \frac{1}{|\mathbb{F}|}\left((2^r-1)\frac{2\eta^5+3\eta\delta\rho}{3\rho^{5/2}} + \frac{r\eta}{\sqrt{\rho}}\right)$$

Proof. The result is a direct consequence of the fact that all the $\mathcal{C}_i$ have the same rate ρ. □

5.4 Numerical Examples

This section follows the methodology used in [Hab22] in order to give examples of parameters guaranteeing a certain security level for FRI. We work in a context similar to that of the paper in order to highlight the gains made possible by the new results of [BSCH+25].
We therefore take $\mathbb{F}_b$, a base field of size 2^{64}, as well as $r = 12$. In this numerical section, we consider the particular case where $|\mathcal{L}_0| = 2^r\rho^{-1} = 2^{12}\rho^{-1}$, so that $d = 2^r$ and therefore $d/2^r = 1$. The purpose of this numerical study is then to identify relevant trade-offs for a concrete implementation of the protocol, that is, to identify choices of parameters $\mathbb{F}$ (an extension of $\mathbb{F}_b$), ρ, δ, and t adapted to a given use case and to the targeted security level.

For a security parameter λ, we then want to check that

$$\varepsilon_C \leq \frac{2^{-\lambda}}{2} \qquad \text{and} \qquad \varepsilon_Q \leq \frac{2^{-\lambda}}{2}$$

thereby guaranteeing that $\varepsilon_{FRI} = \varepsilon_C + \varepsilon_Q \leq 2^{-\lambda}$. The idea is then to start by choosing, for a fixed ρ, the maximal value of δ that guarantees the inequality for ε_C. To do so, we set

$$\forall \delta \in]0, 1-\sqrt{\rho}[, \quad \Gamma(\delta) := (2^r-1)\frac{2\eta^5+3\eta\delta\rho}{3\rho^{5/2}} + \frac{r\eta}{\sqrt{\rho}}$$

which satisfies, for every $\delta \in]0, 1-\sqrt{\rho}[$, the bound $\varepsilon_C \leq \dfrac{\Gamma(\delta)}{|\mathbb{F}|}$.
Thus, guaranteeing the desired inequality amounts to choosing the largest possible δ satisfying the inequality

$$\log_2(\Gamma(\delta)) \leq \log_2(|\mathbb{F}|) - \lambda - 1 \qquad (\mathcal{S}_C)$$

We describe this bound, as a function of different security parameters and field sizes, in the following table:

	$\lambda = 66$	$\lambda = 112$	$\lambda = 128$
$\lvert\mathbb{F}\rvert = 2^{64}$	-3	-49	-65
$\lvert\mathbb{F}\rvert = 2^{128}$	61	15	-1
$\lvert\mathbb{F}\rvert = 2^{192}$	125	79	63

Table 3: Values of $\log_2(|\mathbb{F}|) - \lambda - 1$ as a function of λ and $|\mathbb{F}|$.

Remark. Some parameters are already incompatible. Moreover, it is always easier to guarantee a given security level over large fields. However, one must also take into account the complexity of the computations performed by the Prover, which increases with the field size.
In addition, for each value of λ, the actual security to consider is increased by 14 bits by the *grinding* method presented in [Sta23], which we do not discuss further in this report.

In order to discretize the evolution of δ, we use the notation introduced in [BSCI+20]:

$$\delta = 1 - \left(1 + \frac{1}{2m}\right)\sqrt{\rho}, \qquad \text{with } m \geq 3.$$

Once m has been determined, it then remains to choose, for the number of QUERY steps, the integer t defined by:

$$t = \left\lceil \frac{-(\lambda+1)}{\log_2(1-\delta)} \right\rceil \tag{$\mathcal{S}_Q$}$$

For $\lambda = 66$, there are two possible field sizes. We first carry out the complete study for $|\mathbb{F}| = 2^{128}$ and $\mathcal{R} = 4$ (where $\rho = 2^{-\mathcal{R}}$). We thus numerically determine the largest $m \geq 3$ satisfying ($\mathcal{S}_C$). We find $m = 120$ with

$$\log_2\left[\Gamma\left(1 - \left(1 + \frac{1}{2m}\right)\sqrt{\rho}\right)\right] \approx 60.96$$

We then set $\delta = 1 - \left(1 + \frac{1}{2 \times 120}\right)\sqrt{\rho}$, and determine t in a similar way: we find $t = 34$. We can then fix this value of t and choose the smallest integer m' (and therefore δ) still guaranteeing ($\mathcal{S}_Q$) for this value of t: this increases the soundness of COMMIT without any trade-off. We find numerically $m' = 25$, and therefore

$$\varepsilon_C \lesssim 2^{-78.29} \qquad \text{and} \qquad \varepsilon_Q \lesssim 2^{-67.03}$$

We now present the results obtained for a variety of parameters:

$\mathcal{R}$	m	m'	t
3	170	65	45
4	120	25	34
5	85	39	27

Table 4: Results for $\lambda = 66$ and $|\mathbb{F}| = 2^{128}$.

$\mathcal{R}$	m	m'	t
6	430920	9	23
8	215460	13	17
10	107729	4	14

Table 5: Results for $\lambda = 66$ and $|\mathbb{F}| = 2^{192}$.

$\mathcal{R}$	m	m'	t
6	732	28	38
8	366	7	29
10	182	9	23

Table 6: Results for $\lambda = 112$ and $|\mathbb{F}| = 2^{192}$.

$\mathcal{R}$	m	m'	t
3	225	42	87
4	159	47	65
5	112	38	52

Table 7: Results for $\lambda = 128$ and $|\mathbb{F}| = 2^{192}$.

First, among all the pairs $(\lambda, |\mathbb{F}|)$ considered that seem usable, the pair $(112, 2^{128})$ is missing. This is because the field size does not offer enough margin to ensure the expected security for the COMMIT phase. Indeed, even for the parameters most favorable to this phase, namely $\mathcal{R} = 1$ and $m = 3$, we have

$$\Gamma(\delta) > 2^{22} > 2^{15}$$

The second remark comes from the comparison between the results obtained and those of [Hab22]. We observe that, for identical parameters and configuration, we obtain a slightly smaller number of queries t thanks to the improvement brought by [BSCH+25]. However, this remains a rather small optimization.

5.5 Protocol Variants

The purpose of this section is to present a few variants of the FRI protocol adapted to specific contexts. It contains few technical details and mainly aims to offer an overview of the different existing possibilities. Some of the modifications presented can even be combined depending on the use case.

5.5.1 Batched-FRI: committing to several oracles

In proof systems built from FRI, it is often necessary to check the proximity of several oracles to an RS code. This is in particular the case in the context of the DEEP-ALI arithmetization seen earlier. Naively, this therefore requires as many invocations of the FRI protocol as there are oracles to check. However, it is possible to group these oracles in order to make only a single call to the FRI protocol. This gives the *Batched-FRI* protocol.
We denote by $f_0, \dots, f_L$ oracles on the domain $\mathcal{L}_0$, and then set

$$h = \sum_{\ell=0}^{L} \lambda_\ell \cdot f_\ell$$

for $\lambda_0, \dots, \lambda_L \leftarrow\$ \mathbb{F}$ sent by the Verifier. The Batched-FRI protocol then aims to check that h is close to $\mathrm{RS}[\mathbb{F}, \mathcal{L}_0, d]$, but also that h has indeed been constructed as announced. We obtain:

Phase COMMIT:

1. The Prover sends the oracles $[f_0(x)]_{x\in\mathcal{L}_0},\dots,[f_L(x)]_{x\in\mathcal{L}_0}$.
2. The Verifier chooses batching coefficients $\lambda_0,\dots,\lambda_L\in\mathbb{F}$ and sends them to the Prover.
3. The Prover returns an oracle $[h_0(x)]_{x\in\mathcal{L}_0}$ with
$$h_0(x)=\sum_{\ell=0}^{L}\lambda_\ell f_\ell(x).$$
4. For every round $i=1,\dots,r$:
   (i) The Verifier sends a challenge $\alpha_i\in\mathbb{F}$.
   (ii) The Prover returns an oracle $[h_i(x)]_{x\in\mathcal{L}_i}$ with $h_i=\mathsf{Fold}(h_{i-1},\alpha_i)$.

Phase QUERY:

5. The Verifier chooses $s_1,\dots,s_t\in\mathcal{L}_0$ and sends them to the Prover.
6. The Verifier reads h_r entirely on $\mathcal{L}_r$ and checks that $h_r\in\mathcal{C}_r$.
7. For every $j=1,\dots,t$:
   - $s\leftarrow s_j$.
   - The Verifier queries $f_0,\dots,f_L$ and h_0 at the value s, then checks that
   $$h_0(s)=\sum_{\ell=0}^{L}\lambda_\ell f_\ell(s).$$
   - For every $i=1,\dots,r$:
     (i) The Verifier queries h_{i-1} at the values $s^{2^{i-1}}$ and $-s^{2^{i-1}}$, then computes
     $$\mathsf{Fold}(h_{i-1},\alpha_i)(s^{2^i})=\frac{h_{i-1}(s^{2^{i-1}})+h_{i-1}(-s^{2^{i-1}})}{2}+\alpha_i\frac{h_{i-1}(s^{2^{i-1}})-h_{i-1}(-s^{2^{i-1}})}{2s^{2^{i-1}}}.$$
     (ii) The Verifier queries h_i at the value s^{2^i} and checks that
     $$\mathsf{Fold}(h_{i-1},\alpha_i)(s^{2^i})=h_i(s^{2^i}).$$

PROTOCOL 4 - Batched-FRI

This method gives a considerable complexity gain at the cost of a slight security trade-off, since the results of [BSCI+20] and [BSCH+25] show that

$$\varepsilon_{bFRI}=\varepsilon_{Batch}+\varepsilon_{FRI}\qquad\text{with }\varepsilon_{Batch}=\varepsilon_{ca}(\mathcal{C}_0,\delta).$$

Moreover, one can still trade part of this soundness for a reduction in the number of interactions. To do this, one samples a single $\lambda\leftarrow\$\,\mathbb{F}$ and sets $\lambda_\ell=\lambda^\ell$ for every $\ell\in\{0,\dots,L\}$.
This configuration gives $\varepsilon'_{Batch}=L\cdot\varepsilon_{Batch}$ and plays a central role in setting up the BCS technique for DEEP-ALI/FRI, as described in [BSCS16] and [BGK+23].

5.5.2 *DEEP-FRI: an obsolete soundness improvement*

Following the publication of FRI in [BSBHR18a] in 2018, the DEEP-FRI variant was introduced as early as 2019 in [BSGKS19]. As indicated earlier in this report, this publication introduced many applications of the DEEP technique, including this new protocol. It remains very close to FRI and mainly introduces modifications during the COMMIT phase. It consists of replacing, at each step, the commitments to the oracles by an instance of the PCS described in 1:

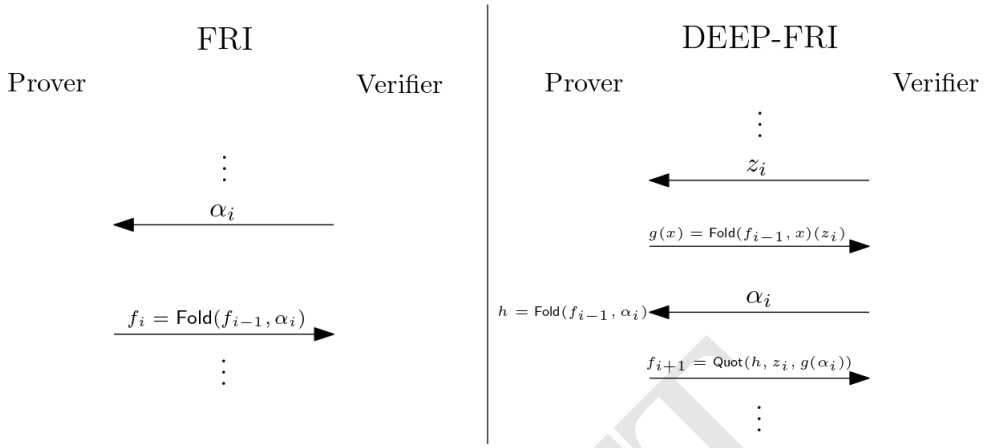


Figure 3: Description of DEEP-FRI

When it was introduced, DEEP-FRI seemed to improve the proven soundness of FRI by making it possible to choose a proximity parameter up to the Johnson bound. The results of [BSCI+20] nevertheless showed that this bound is already reached by FRI itself, without modifying the protocol. The main contribution of DEEP-FRI therefore now appears mostly historical and methodological, through the introduction of the DEEP technique, rather than as a lasting practical improvement of FRI as a proximity test.

5.5.3 *FRI-Binius & Circle-STARKs: extending the FFT*

In 2024, two new variants of FRI were studied and developed in parallel, with the same goal but following two different approaches. Both aim to extend the choice of the base field for FRI. Indeed, among the initial assumptions required to build a complete proof system, the constraint of having an *FFT-friendly* field appears to be a major one. Thus, for recurring interpolation needs, it seems initially necessary to choose a base field $\mathbb{F}$ allowing the existence of a multiplicative subgroup of order 2^k, with k large enough, and therefore, at the same time, the existence of a 2^k-th root of unity. This greatly restricts the choice of base fields and in particular excludes some fields over which computations are more efficient. We therefore briefly present these two variants:

- *StarkWare* develops its proof system *Stwo* from its *Circle-FRI* protocol described in [HLP24]. This approach relies on the fact that the field $M31$, defined as $\mathbb{F}_p$ for $p = 2^{31} - 1$, is extremely well suited to 32-bit architectures, while not being *FFT-friendly*. The idea is then to base the arithmetization on cosets of cyclic subgroups of the unit circle $C(\mathbb{F}_p) = \{(x,y) \in \mathbb{F}_p \mid x^2 + y^2 = 1\}$, endowed with its group structure. The starting point of the approach is the observation that, for any extension $\tilde{\mathbb{F}}$ of $\mathbb{F}_p$, the group $C(\tilde{\mathbb{F}})$ contains a cyclic subgroup of order 2^{30}. This structure then serves as the basis for a variant of the FFT, called *Circle-FFT*, and then for the *Circle-FRI* protocol.

- *Irreducible* proposes in [DP24] its generic *FRI-Binius* protocol, based on [ZCF23], which makes it possible to adapt FRI to binary fields $\mathbb{F}$. By nature, these fields are usually excluded since

$|\mathbb{F}^*| = 2^k - 1$ is odd. They then use a new approach, called *Additive-FFT*, presented in [LCH14], which makes it possible to apply the technique to these same binary fields by exploiting their structure as $\mathbb{F}_2$-vector spaces. The use of these fields allows a more natural arithmetization for bitwise operations and can therefore offer shorter proving times on average.

REFERENCES

[ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed–solomon proximity testing with super-fast verification. Cryptology ePrint Archive, Paper 2024/1586, 2024.

[BCCT11] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. Cryptology ePrint Archive, Paper 2011/443, 2011.

[BGK+23] Alexander R. Block, Albert Garreta, Jonathan Katz, Justin Thaler, Pratyush Ranjan Tiwari, and Michał Zając. Fiat-shamir security of FRI and related SNARKs. Cryptology ePrint Archive, Paper 2023/1071, 2023.

[BSBHR18a] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon Interactive Oracle Proofs of Proximity, 2018.

[BSBHR18b] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046, 2018.

[BSCH+25] Eli Ben-Sasson, Dan Carmon, Ulrich Haböck, Swastik Kopparty, and Shubhangi Saraf. On proximity gaps for reed–solomon codes. Cryptology ePrint Archive, Paper 2025/2055, 2025.

[BSCI+20] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for reed-solomon codes. Cryptology ePrint Archive, Paper 2020/654, 2020.

[BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. Cryptology ePrint Archive, Paper 2016/116, 2016.

[BSGKS19] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. DEEP-FRI: Sampling outside the box improves soundness. Cryptology ePrint Archive, Paper 2019/336, 2019.

[CY24] Alessandro Chiesa and Eylon Yogev. *Building Cryptographic Proofs from Hash Functions*. 2024.

[DP24] Benjamin E. Diamond and Jim Posen. Polylogarithmic proofs for multilinears over binary towers. Cryptology ePrint Archive, Paper 2024/504, 2024.

[Fu25] Shihui Fu. Improved constant-sized polynomial commitment schemes without trusted setup. Cryptology ePrint Archive, Paper 2025/1233, 2025.

[GMW25] Albert Garreta, Nicolas Mohnblatt, and Benedikt Wagner. A simplified round-by-round soundness proof of FRI. Cryptology ePrint Archive, Paper 2025/1993, 2025.

[Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In *Proceedings, Part II, of the 35th Annual International Conference on Advances in Cryptology — EUROCRYPT 2016 - Volume 9666*, page 305–326. Springer-Verlag, 2016.

[Gur04] Venkatesan Guruswami. *List Decoding of Error-Correcting Codes: Winning Thesis of the 2002 ACM Doctoral Dissertation Competition*, volume 3282 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 2004.

[Gur07] Venkatesan Guruswami. *Algorithmic results in list decoding*, volume 2(2) of *Foundations and Trends® in Theoretical Computer Science*. Now Publishers Inc., 2007.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Paper 2019/953, 2019.

[Hab22] Ulrich Haböck. A summary on the FRI low degree test. Cryptology ePrint Archive, Paper 2022/1216, 2022.

[HLP24] Ulrich Haböck, David Levit, and Shahar Papini. Circle STARKs. Cryptology ePrint Archive, Paper 2024/278, 2024.

[KPV19] Assimakis Kattis, Konstantin Panarin, and Alexander Vlasov. RedShift: Transparent SNARKs from list polynomial commitments. Cryptology ePrint Archive, Paper 2019/1400, 2019.

[KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *Advances in Cryptology - ASIACRYPT 2010*, pages 177–194. Springer Berlin Heidelberg, 2010.

[LCH14] Sian-Jheng Lin, Wei-Ho Chung, and Yunghsiang S. Han. Novel polynomial basis and its application to reed-solomon erasure codes. In *2014 IEEE 55th Annual Symposium on Foundations of Computer Science*, pages 316–325, 2014.

[MF23] Tiago Martins and João Farinha. Study of arithmetization methods for STARKs. Cryptology ePrint Archive, Paper 2023/661, 2023.

[SB25a] Whiteboard Sessions and Dan Boneh. Zk whiteboard sessions - S2M7: FRI and proximity proofs (part 1), 2025.

[SB25b] Whiteboard Sessions and Dan Boneh. Zk whiteboard sessions - S2M7: FRI and proximity proofs (part 2), 2025.

[Sta23] StarkWare. ethSTARK documentation – version 1.2. Cryptology ePrint Archive, Paper 2021/582, 2023.

[ZCF23] Hadas Zeilberger, Binyi Chen, and Ben Fisch. BaseFold: Efficient field-agnostic polynomial commitment schemes from foldable codes. Cryptology ePrint Archive, Paper 2023/1705, 2023.

A SCHWARTZ-ZIPPEL LEMMA

An important property that often appears in the construction of sound proof systems is the *Schwartz-Zippel lemma*. In an adapted context, this result allows a Verifier to compare two polynomials with only one evaluation query to each polynomial:

Lemma A.1 (Schwartz-Zippel)**.** *Let $\mathbb{F}$ be a field and let $P \in \mathbb{F}[X_1, \ldots, X_n]$ be nonzero of degree d. Let $S \subset \mathbb{F}$ be nonempty and let $s_1, \ldots, s_n \in S$ be drawn uniformly and independently from S. Then,*

$$\mathop{\mathbb{P}}_{s_i}[P(s_1, \ldots, s_n) = 0] \leq \frac{d}{|S|}.$$

Proof. For $n = 1$, the property is clear since P has at most d roots.

We assume the lemma is true for every $k < n$. We write $P = \sum_{i=0}^{d} X_1^i Q_i(X_2, \ldots, X_n)$. By the assumption on P, there necessarily exists a Q_i that is nonzero, and we set

$$r = \max\{i \ , \ Q_i \neq 0\}.$$

We then have $\deg Q_r \leq d - r$ and therefore the following facts:

- $\mathop{\mathbb{P}}_{s_j}[Q_r(s_2, \ldots, s_n) = 0] \leq \frac{d-r}{|S|}$;
- If $Q_r(s_2, \ldots, s_n) \neq 0$, then $P(X_1, s_2, \ldots, s_n)$ is a univariate polynomial of degree r.

We define the events A: "$P(s_1, \ldots, s_n) = 0$" and B: "$Q_r(s_2, \ldots, s_n) = 0$".
The preceding facts then give:

$$\mathbb{P}[B] \leq \frac{d-r}{|S|} \qquad \text{and} \qquad \mathbb{P}\left[A \mid \overline{B}\right] \leq \frac{r}{|S|}.$$

We then conclude by

$$\mathbb{P}[A] = \mathbb{P}[A \mid B]\mathbb{P}[B] + \mathbb{P}\left[A \mid \overline{B}\right]\mathbb{P}\left[\overline{B}\right] \leq \mathbb{P}[B] + \mathbb{P}\left[A \mid \overline{B}\right] \leq \frac{d-r}{|S|} + \frac{r}{|S|} = \frac{d}{|S|}.$$

□

Corollary A.2. *Let $\mathbb{F}$ be a finite field and let $P \in \mathbb{F}[X_1, \ldots, X_n]$ be nonzero of degree d. Then, for $s_1, \ldots, s_n$ drawn uniformly and independently from $\mathbb{F}$, we have*

$$\mathop{\mathbb{P}}_{s_i}[P(s_1, \ldots, s_n) = 0] \leq \frac{d}{|\mathbb{F}|}.$$

In particular, this corollary shows that for two distinct polynomials $P, Q \in \mathbb{F}^{<d}[X]$ and for z drawn uniformly from $\mathbb{F}$, we have $P(z) = Q(z)$ with probability at most $\frac{d}{|\mathbb{F}|}$.

B BERLEKAMP-WELCH ALGORITHM

In this section, we present the Berlekamp-Welch algorithm for decoding Reed-Solomon codes. In other words, in the unique decoding regime, it determines in polynomial time the unique bounded-degree polynomial close to an oracle on a fixed domain.

We fix $\mathcal{C} = \mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$, a proximity parameter $0 < \delta \leq \frac{1-\rho}{2}$, and an oracle $g : \mathcal{L} \longrightarrow \mathbb{F}$ such that

$\Delta(g,\mathcal{C}) \leq \delta$. The goal is then to reconstruct the unique polynomial $f \in \mathbb{F}^{<d}[X]$ such that $\Delta(g,f) \leq \delta$. We write $n = |\mathcal{L}|$ and $t = \lfloor n\delta \rfloor$ for the maximum number of positions where g and f may differ.

Localization. We construct the **error-locator polynomial** E by:

$$E(X) = \prod_{\substack{z \in \mathcal{L} \\ f(z) \neq g(z)}} (X - z) \neq 0.$$

We note that determining E is as hard as determining f directly, since it would then suffice to interpolate on the points that do not appear among the roots of E. However, this polynomial will allow us to rewrite the initial problem. Indeed, for every $z \in \mathcal{L}$, we have

$$f(z) = g(z) \qquad \text{or} \qquad E(z) = 0.$$

Thus, determining f amounts to solving

$$\begin{cases} \forall z \in \mathcal{L}, & f(z)E(z) = g(z)E(z), \\ \deg f < d & \text{and} \quad \deg E \leq t. \end{cases} \tag{$\mathcal{E}$}$$

However, this is a nonlinear system, and therefore difficult to solve.

Linearization. We set $F(X) = f(X)E(X) \in \mathbb{F}^{<d+t}[X]$ and

$$\begin{cases} \forall z \in \mathcal{L}, & F(z) = g(z)E(z), \\ \deg F < t + d & \text{and} \quad \deg E \leq t. \end{cases} \tag{$\mathcal{E}'$}$$

($\mathcal{E}'$) has the advantage of being a linear system consisting of n equations and a number of unknowns bounded by

$$t + d + t + 1 \leq 2t + d + 1 \leq n + 1.$$

Moreover, we know that E is monic, so in reality we have at most n unknowns. However, even though it is clear that ($\mathcal{E}$) $\implies$ ($\mathcal{E}'$), the converse implication is less obvious. We nevertheless have the following lemma:

Lemma B.1. *If $(E,F) \neq (0,0)$ is a solution of ($\mathcal{E}'$), then $\dfrac{F}{E} \in \mathbb{F}^{<d}[X]$ and this polynomial is a solution of ($\mathcal{E}$).*

Proof. Let (E_1,F_1) and (E_2,F_2) be two pairs of solutions of ($\mathcal{E}'$). Then

$$\forall z \in \mathcal{L}, \qquad E_1(z)F_2(z) = g(z)E_1(z)E_2(z) = E_2(z)F_1(z).$$

In particular, $E_1(X)F_2(X)$ and $E_2(X)F_1(X)$ coincide on n values, and their respective degrees are strictly bounded by $2t + d \leq n$. Hence these polynomials are equal.
Now let $E_\star$ be the error locator associated with the decoded polynomial f. Then $(E_\star, fE_\star)$ is a solution of ($\mathcal{E}'$). For any other nonzero solution (E,F) of ($\mathcal{E}'$), we therefore obtain

$$E_\star(X)F(X) = E(X)f(X)E_\star(X).$$

Since $\mathbb{F}[X]$ is an integral domain and $E_\star \neq 0$, we deduce that $f = \dfrac{F}{E} \in \mathbb{F}^{<d}[X]$. □

We can now describe the Berlekamp-Welch algorithm below.

Algorithm 2 Berlekamp-Welch decoding for $\mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$

Require: $g \in \mathbb{F}^{\mathcal{L}}$ such that $\Delta(g, \mathcal{C}) \leq \delta$
Ensure: $f \in \mathbb{F}^{<d}[X]$ such that $\Delta(g, f) \leq \delta$
1: Construct the linear system $(\mathcal{E}')$ from g, d, and t.
2: Determine a pair of solutions (E, F) of $(\mathcal{E}')$ with E monic.
3: $f \leftarrow \dfrac{F}{E}$
4: **if** $f \notin \mathbb{F}^{<d}[X]$ or $\Delta(g, f) > \delta$ **then**
5: **return** $\perp$
6: **end if**
7: **return** f

Remark. The preceding lemma guarantees that any nonzero solution of $(\mathcal{E}')$ determines the same quotient $\dfrac{F}{E}$. Thus, as soon as g is at relative distance at most δ from the code $\mathcal{C}$, the algorithm reconstructs the corresponding unique polynomial $f \in \mathbb{F}^{<d}[X]$. Conversely, if the algorithm returns $\perp$, then there is no polynomial of degree strictly less than d at relative distance at most δ from g, that is, $\Delta(g, \mathcal{C}) > \delta$.

In terms of complexity, the dominant step is solving the linear system $(\mathcal{E}')$, which has $O(n)$ equations and $O(n)$ unknowns. We therefore obtain an algorithm polynomial in n, for instance in $O(n^3)$ operations over $\mathbb{F}$ with naive Gaussian elimination. The other steps, essentially polynomial Euclidean division and the final check over the domain $\mathcal{L}$, have at most quadratic cost at this level of precision and are therefore dominated by solving the system.

C BIT-REVERSAL NUMBERING

After presenting FRI, it is useful to describe an optimization for implementing the protocol. It relies only on a suitable indexing of the initial domain ensuring that, in the binary tree built from the domain, *the values attached to sibling nodes and sibling leaves have the same square, this square being the value attached to the parent node*. This property follows directly from the result below:

Proposition C.1. *Let $\mathcal{L} = \langle w \rangle \subseteq \mathbb{F}^{\times}$ be a multiplicative subgroup of cardinality 2^k. For every $i \in \{0, \ldots, 2^k - 1\}$, set*

$$x_i := w^{\mathrm{Renv}_k(i)}.$$

Then, for all $j \in \{0, \ldots, k\}$, $q \in \{0, \ldots, 2^{k-j} - 1\}$, and $i \in \{q2^j, \ldots, (q+1)2^j - 1\}$, we have

$$x_i^{2^j} = w^{2^j \mathrm{Renv}_{k-j}(q)}.$$

Before proving this result, let us show that it indeed makes it possible to construct the desired binary tree. For this, it suffices to observe that

$$\forall j \in \{0, \ldots, k\}, \quad \mathcal{L}_j = \langle w^{2^j} \rangle.$$

Let $j > 0$. For every $i \in \{0, \ldots, 2^k - 1\}$ and every $q \in \{0, \ldots, 2^{k-j} - 1\}$, write:

$$x_i := w^{\mathrm{Renv}_k(i)} \qquad \text{and} \qquad y_q := (w^{2^j})^{\mathrm{Renv}_{k-j}(q)}.$$

We therefore have indexings of $\mathcal{L}_0$ and $\mathcal{L}_j$, given respectively by the x_i and the y_q. The proposition then shows that, for every index i which, by going up the tree, leads to the index q, we have $x_i^{2^j} = y_q$, which is exactly the desired property for this binary tree.

Proof (Proposition C.1). Fix $j \in \{0, \ldots, k\}$ and $q \in \{0, \ldots, 2^{k-j} - 1\}$, and set

$$I_{j,q} = \{q2^j, \ldots, (q+1)2^j - 1\}.$$

For every $i \in I_{j,q}$, we can write $i = q2^j + t$ with $0 \leq t < 2^j$. If $[\cdot]_2$ denotes the binary representation of an integer, padded to the indicated length, then:

$$[i]_2 = [q]_2 \parallel [t]_2,$$

with $[i]_2$ of length k, $[t]_2$ of length j, and $[q]_2$ of length $k-j$. Thus,

$$[\mathrm{Renv}_k(i)]_2 = [\mathrm{Renv}_j(t)]_2 \parallel [\mathrm{Renv}_{k-j}(q)]_2.$$

We then obtain $\mathrm{Renv}_k(i) = 2^{k-j}\mathrm{Renv}_j(t) + \mathrm{Renv}_{k-j}(q)$. Consequently,

$$x_i^{2^j} = w^{2^j \mathrm{Renv}_k(i)} = w^{2^j(2^{k-j}\mathrm{Renv}_j(t) + \mathrm{Renv}_{k-j}(q))} = \left(w^{2^k}\right)^{\mathrm{Renv}_j(t)} w^{2^j \mathrm{Renv}_{k-j}(q)}.$$

Since $|\mathcal{L}| = 2^k$, we have $w^{2^k} = 1$, and therefore

$$x_i^{2^j} = w^{2^j \mathrm{Renv}_{k-j}(q)}.$$

□